

邊緣人的好朋友 在 Discord 跟 AI 機器人聊天吧！

▶ NTNU CSIE CAMP

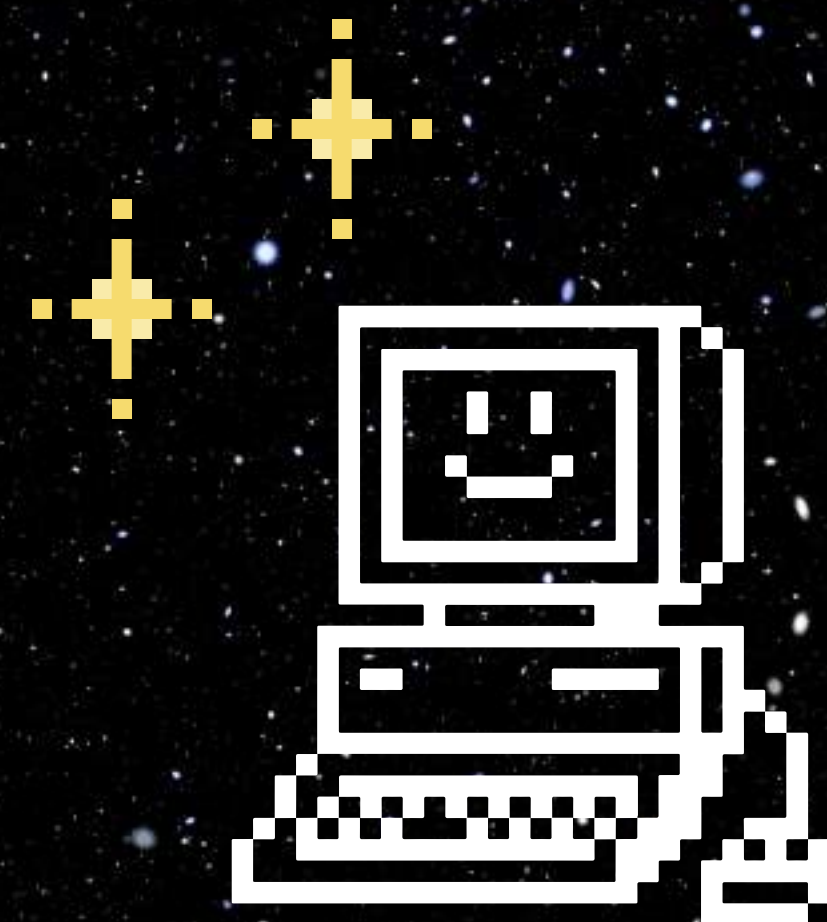
▶ 講者：吐司

本課堂 Slido

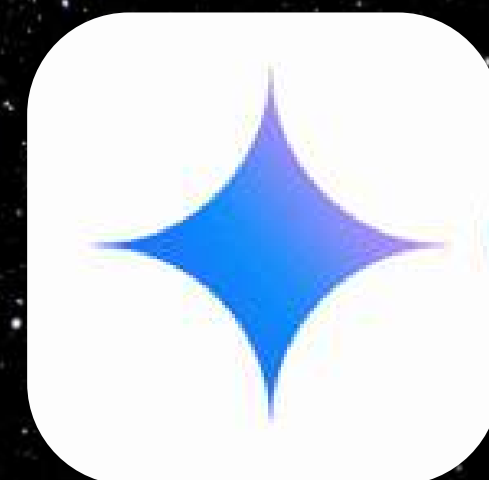


這節課你將學到

- ▶ 生成式 AI 是什麼
- ▶ API 的原理
- ▶ 製作屬於自己的 discord 聊天機器人



有沒有用過
生成式 AI？



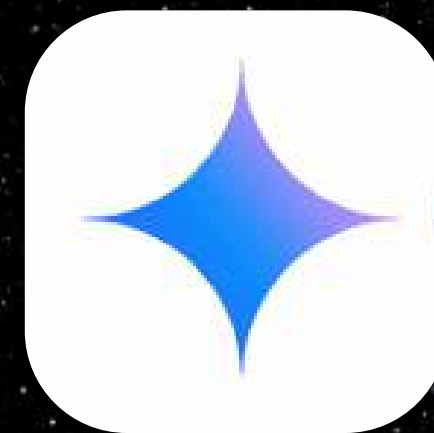
有沒有用過
Discord?



當 Discord
遇到 Gemini
會發生什麼？



+

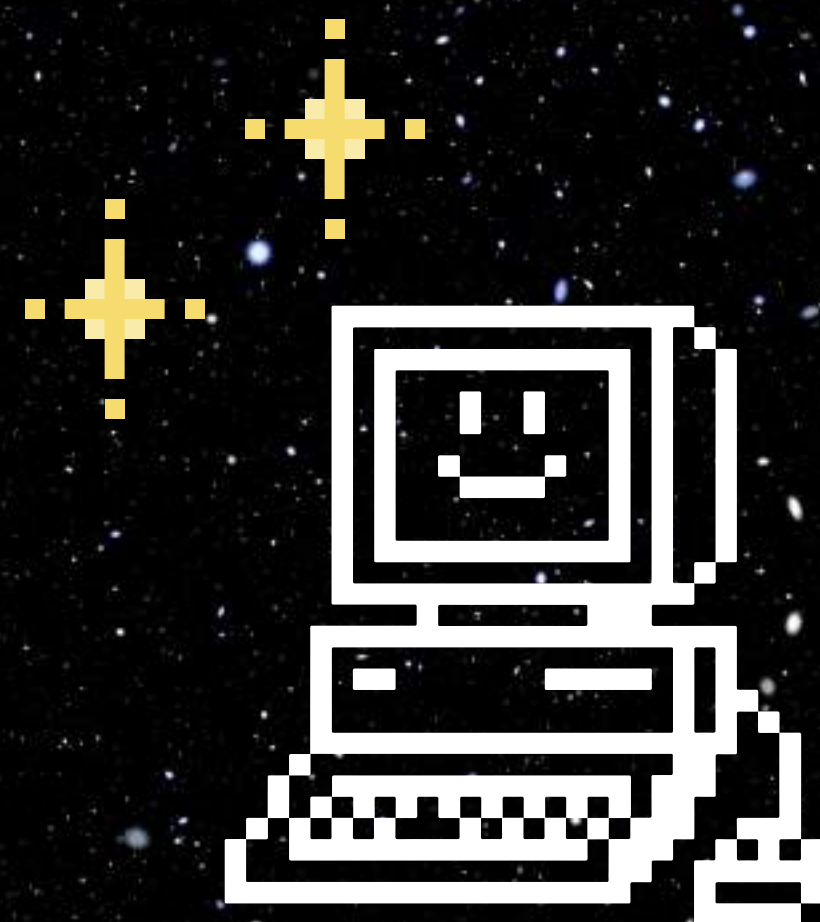


=

?

成果展示描述

- ▶ 使用 Discord bot 串接 Gemini api，製作簡單的對話機器人
- ▶ Gemini 是一款由 Google 開發的生成式 AI



Discord bot

實作

讓我們來做第一隻 Discord bot 吧



Discord bot

- ▶ 是一種可以能夠**自動化處理任務**的機器人
人例如：點播音樂、管理伺服器等
- ▶ 本課程將教你製作一個聊天機器人
- ▶ 之後黑客松也會用到 Discord bot 哦

設立 Discord bot 的預先準備

- ▶ 建立 Discord 伺服器
- ▶ 建立 application
- ▶ 撰寫 Discord bot 程式碼

建立 Discord 伺服器

Discord bot 要放在伺服器裡面

建立 Discord 伺服器

▶ 首先先前往 [Discord 網頁](#)，並登入 Discord 帳號



登入 Discord 帳號

► 選擇習慣的登入方式(推薦使用 QR code)

歡迎回來！

我們很高興又見到您了！

電子郵件或電話號碼 *

密碼 *

[忘記您的密碼？](#)

登入

[需要一個帳號？](#) [註冊](#)



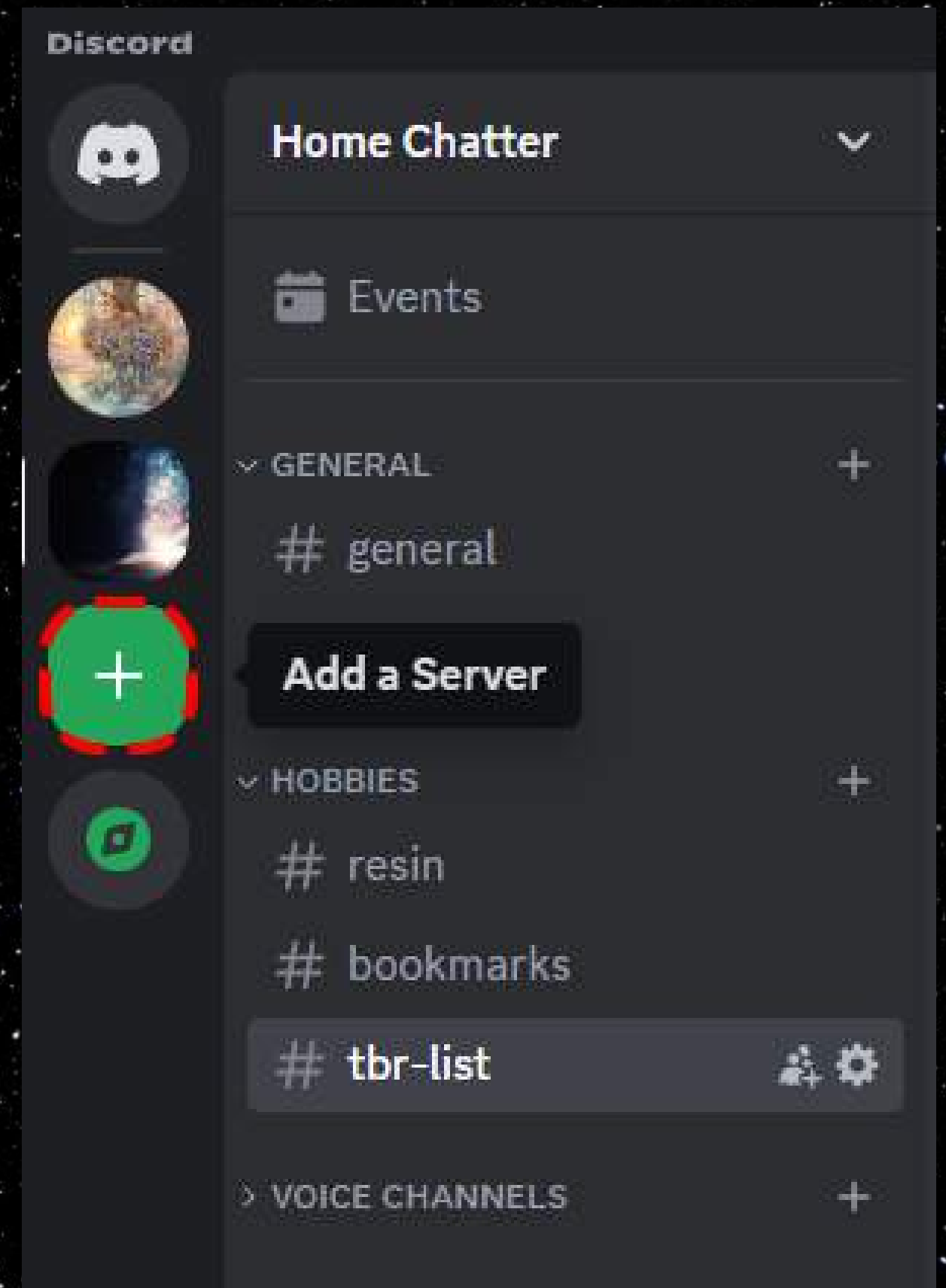
使用 QR Code 登入

用 Discord 行動應用程式 對此掃描
就能立即登入。

或者，使用密碼金鑰登入

建立 Discord 伺服器

- ▶ 在你的帳號中開一個新伺服器
- ▶ 在 Discord 頁面左側中找到 + 號，按下後便可建立伺服器



建立 Discord 伺服器

▶ 按下**建立自己的**



建立 Discord 伺服器

▶ 按下我和我的好友

×

多介紹一下您的伺服器

為了協助您進行設定，請您說明一下，您的新伺服器要供幾位好友還是比較大型的社群使用？

 俱樂部或社群 >

 我和我的好友 >

不太確定嗎？您可以暫時 [跳過這個問題](#)。

上一頁

建立 Discord 伺服器

► 取伺服器名稱並建立伺服器

✕

自訂伺服器

幫您的新伺服器取個名字和選個圖示，您之後隨時都可以變更。



伺服器名稱

的伺服器

在創立伺服器的同時將代表您已同意 Discord 的**社群守則**。

上一頁

建立

設立 Discord bot 的預先準備

- ▶ 建立 Discord 伺服器
- ▶ 建立 application
- ▶ 撰寫 Discord bot 程式碼

建立 application

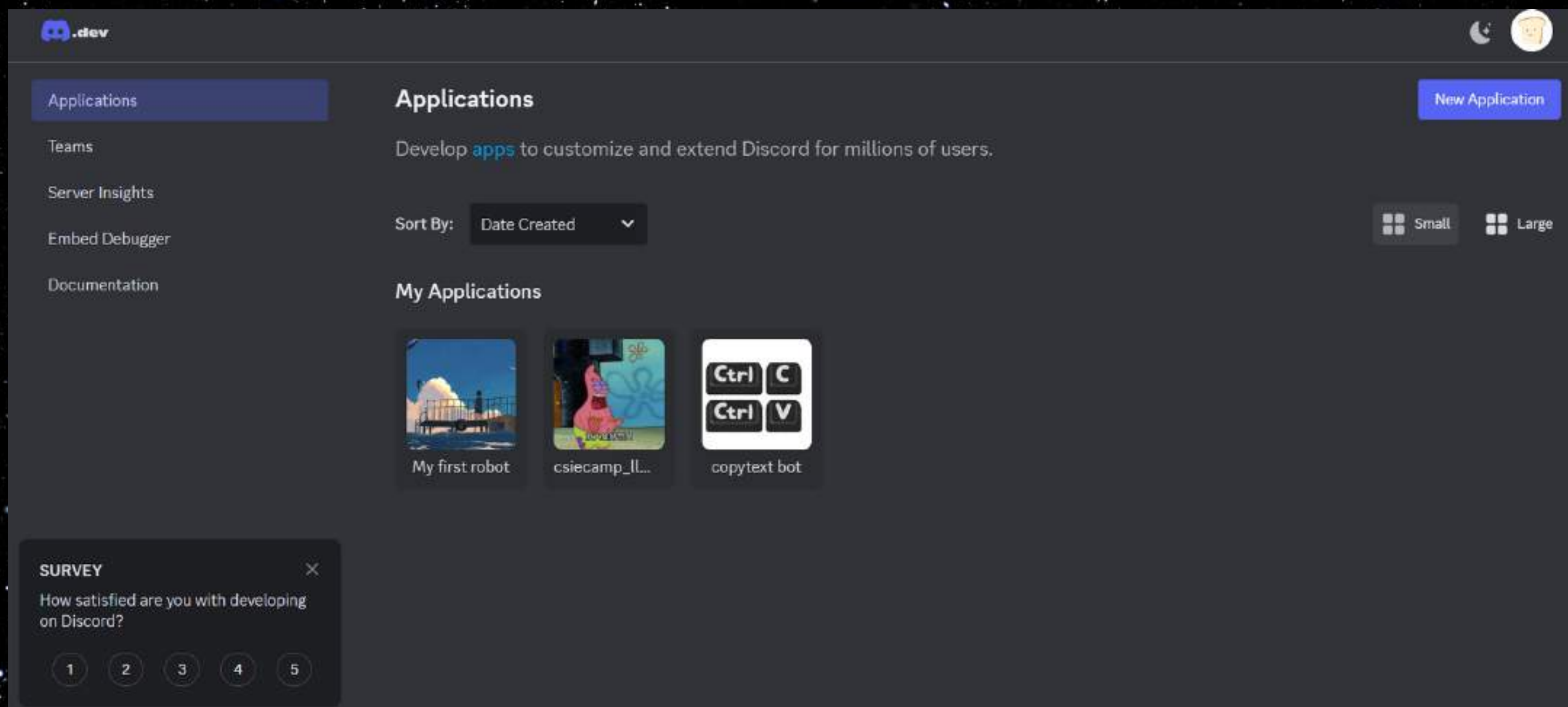
建立 application 讓 discord 知道你要做機器人

為什麼要建立 application

- ▶ 在 Discord developer 網站上建立 application，讓 Discord 知道有這個 Discord bot 存在
- ▶ 之後可以從 application 上面取得 discord bot token

建立 Discord 伺服器

➡ 前往 [Discord Developers](#).



登入 Discord 帳號

► 選擇習慣的登入方式(推薦使用 QR code)

歡迎回來！

我們很高興又見到您了！

電子郵件或電話號碼 *

密碼 *

[忘記您的密碼？](#)

登入

[需要一個帳號？](#) [註冊](#)



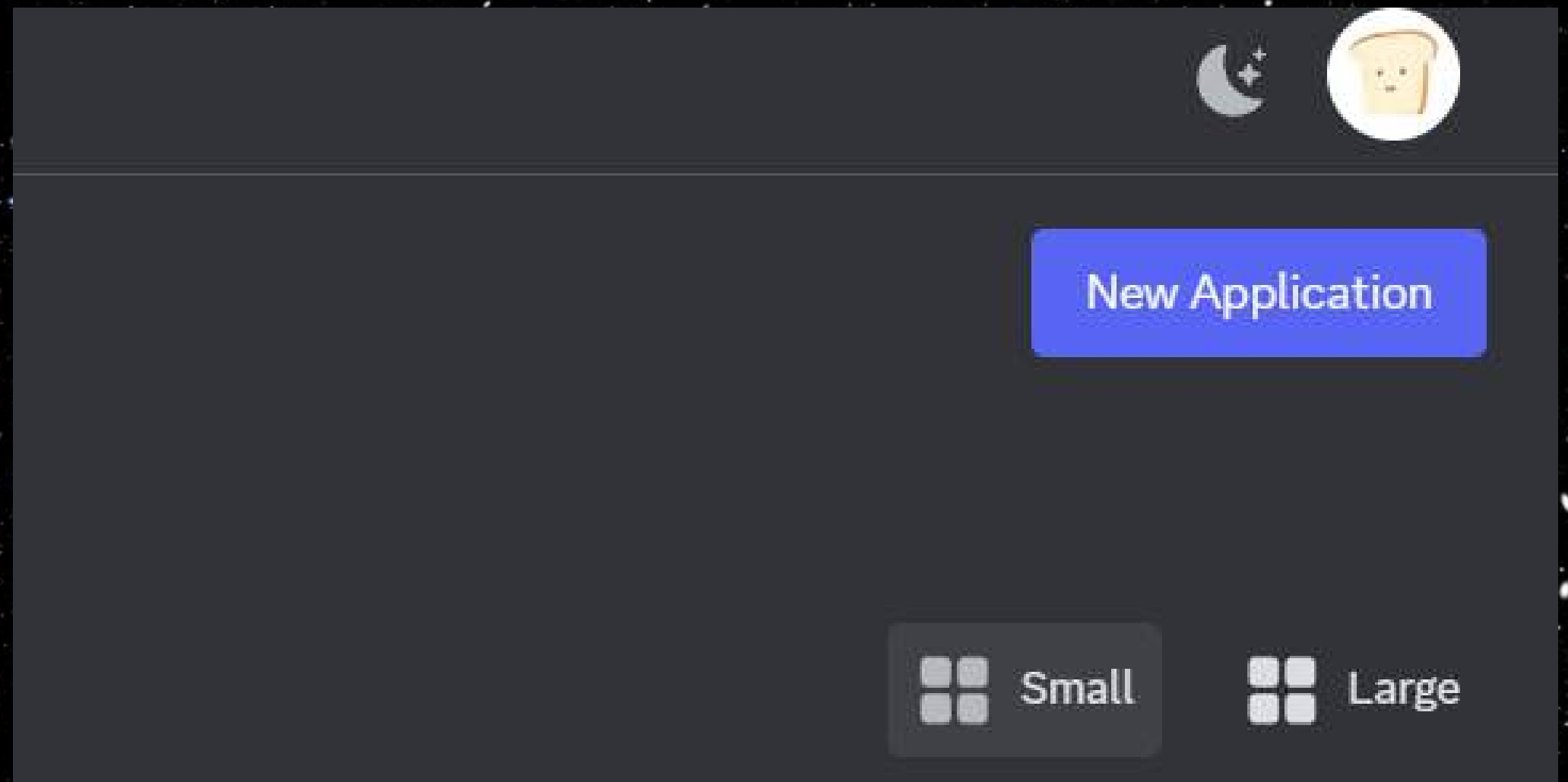
使用 QR Code 登入

用 Discord 行動應用程式 對此掃描
就能立即登入。

或者，使用密碼金鑰登入

建立 Discord 伺服器

▶ 點擊右上角的 **New Application**。



建立 Discord 伺服器

- ▶ 點擊 New Application，創建機器人並設立名字
- ▶ 名字可以自己取

CREATE AN APPLICATION

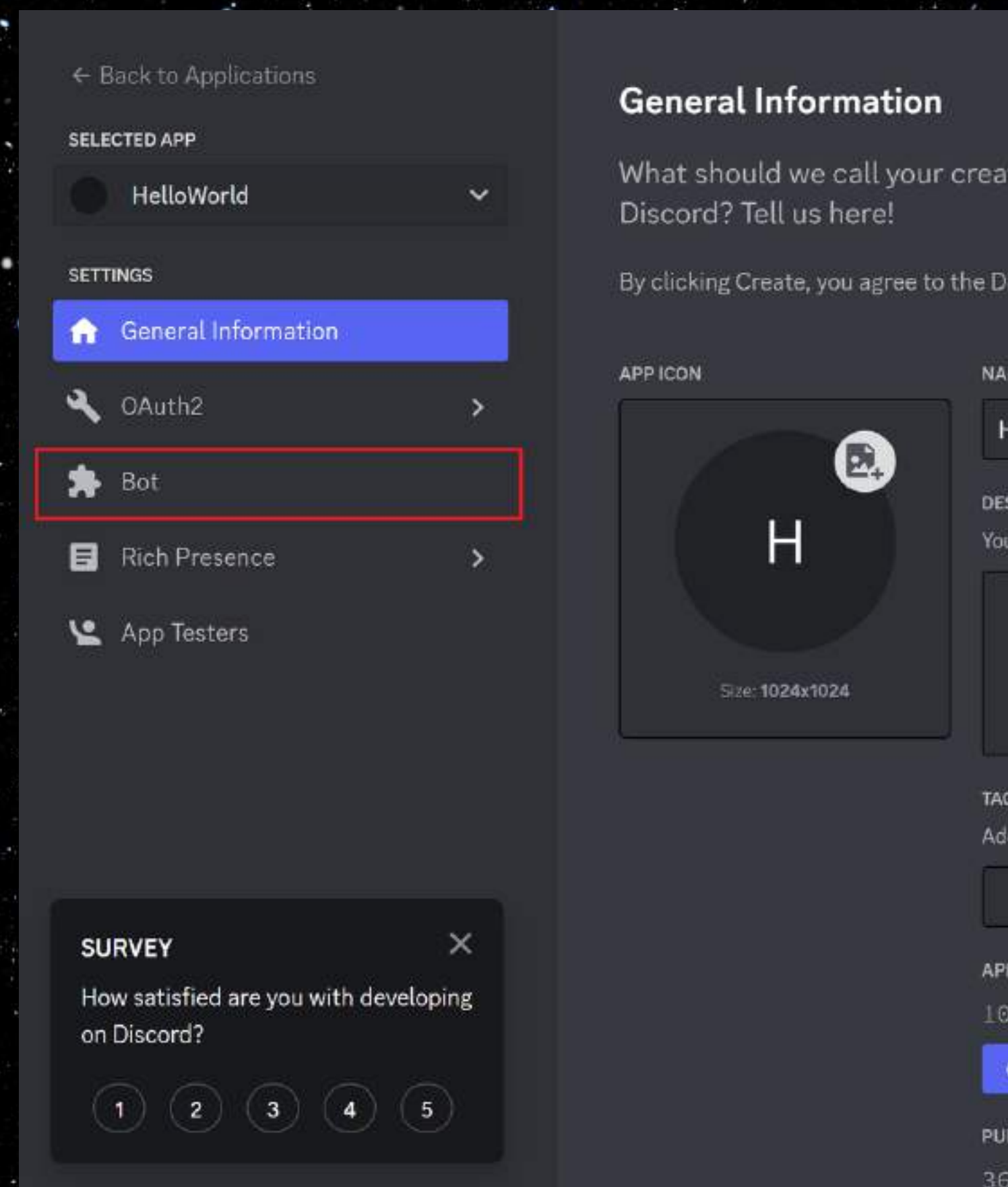
NAME *

☒ By clicking Create, you agree to the Discord [Developer Terms of Service](#) and [Developer Policy](#).

CancelCreate

建立 Discord bot

▶ 點選左側功能欄中的
Bot 進入創建頁面



建立 Discord bot

▶ 點擊 Add Bot 創建機器人身分

← Back to Applications

SELECTED APP

● HelloWorld ▾

SETTINGS

- 🏠 General Information
- 🔧 OAuth2 >
- 🧩 Bot
- 📄 Rich Presence >
- 👤 App Testers

Bot



Bring your app to life on Discord with a Bot user. Be a part of chat in your users' servers and interact with them directly.

[Learn more about bot users](#)

Build-A-Bot

Bring your app to life by adding a bot user. This action is irreversible (because robots are too cool to destroy).

Add Bot



建立 Discord bot

▶ 按下 Yes, do it! 即可

ADD A BOT TO THIS APP?

Adding a bot user gives your app visible life in Discord.
However, this action is irrevocable! Choose wisely.

Nevermind

Yes, do it!

建立 Discord bot

► 往下滑到 **Privileged Gateway Intents** 區塊

REQUIRES OAUTH2 CODE GRANT

If your application requires multiple scopes then you may need the full OAuth2 flow to ensure a bot doesn't join before your application is granted a token.



Privileged Gateway Intents

Some [Gateway Intents](#) require approval if your bot is verified. If your bot is not verified, you can toggle those intents below to access them.

PRESENCE INTENT

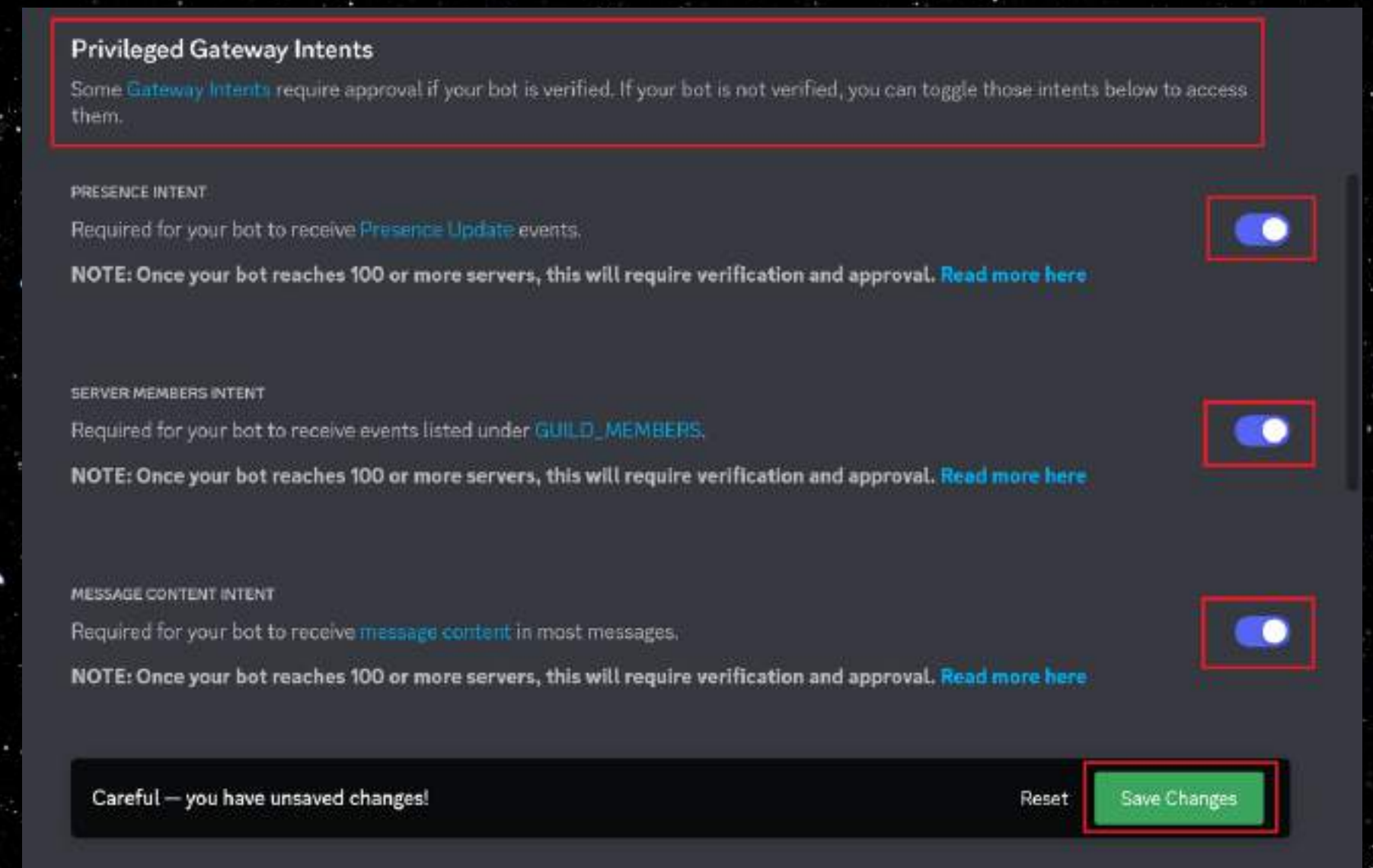
Required for your bot to receive [Presence Update](#) events.



NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)

建立 Discord bot

- ▶ 把三個功能都點選並按下 **Save Changes**
- ▶ 把權限都打開對未來製作機器人比較方便



The screenshot shows the 'Privileged Gateway Intents' section of the Discord bot settings. It contains three toggle switches, all of which are turned on and highlighted with red boxes. The first toggle is for 'Presence Intent', the second for 'Server Members Intent', and the third for 'Message Content Intent'. Each toggle has a corresponding note about reaching 100 servers. At the bottom, there is a 'Save Changes' button, also highlighted with a red box, and a 'Reset' button. A warning message 'Careful — you have unsaved changes!' is displayed at the bottom left.

Privileged Gateway Intents
Some [Gateway Intents](#) require approval if your bot is verified. If your bot is not verified, you can toggle those intents below to access them.

PRESENCE INTENT
Required for your bot to receive [Presence Update](#) events.
NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)

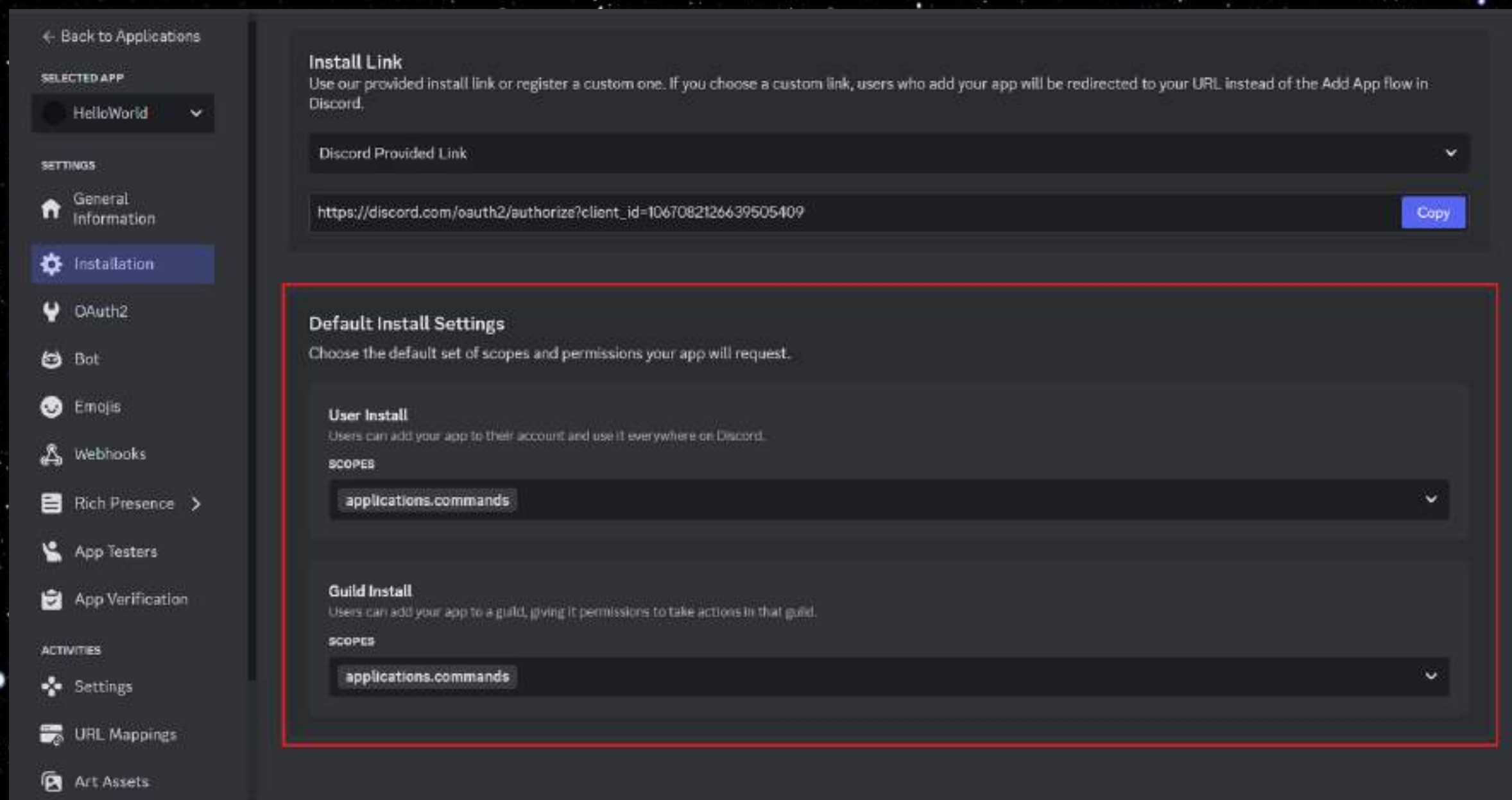
SERVER MEMBERS INTENT
Required for your bot to receive events listed under [GUILD_MEMBERS](#).
NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)

MESSAGE CONTENT INTENT
Required for your bot to receive [message content](#) in most messages.
NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)

Careful — you have unsaved changes! [Reset](#) [Save Changes](#)

建立 Discord bot

▶ 點擊左側功能欄的 **Installation**



建立 Discord bot

- ▶ 在 Guild install 的 scopes 中找到 bot 選項並加入
- ▶ 並按下 save change



Guild Install
Users can add your app to a guild, giving it permissions to take actions in that guild.

SCOPES

applications.commands bot

PERMISSIONS

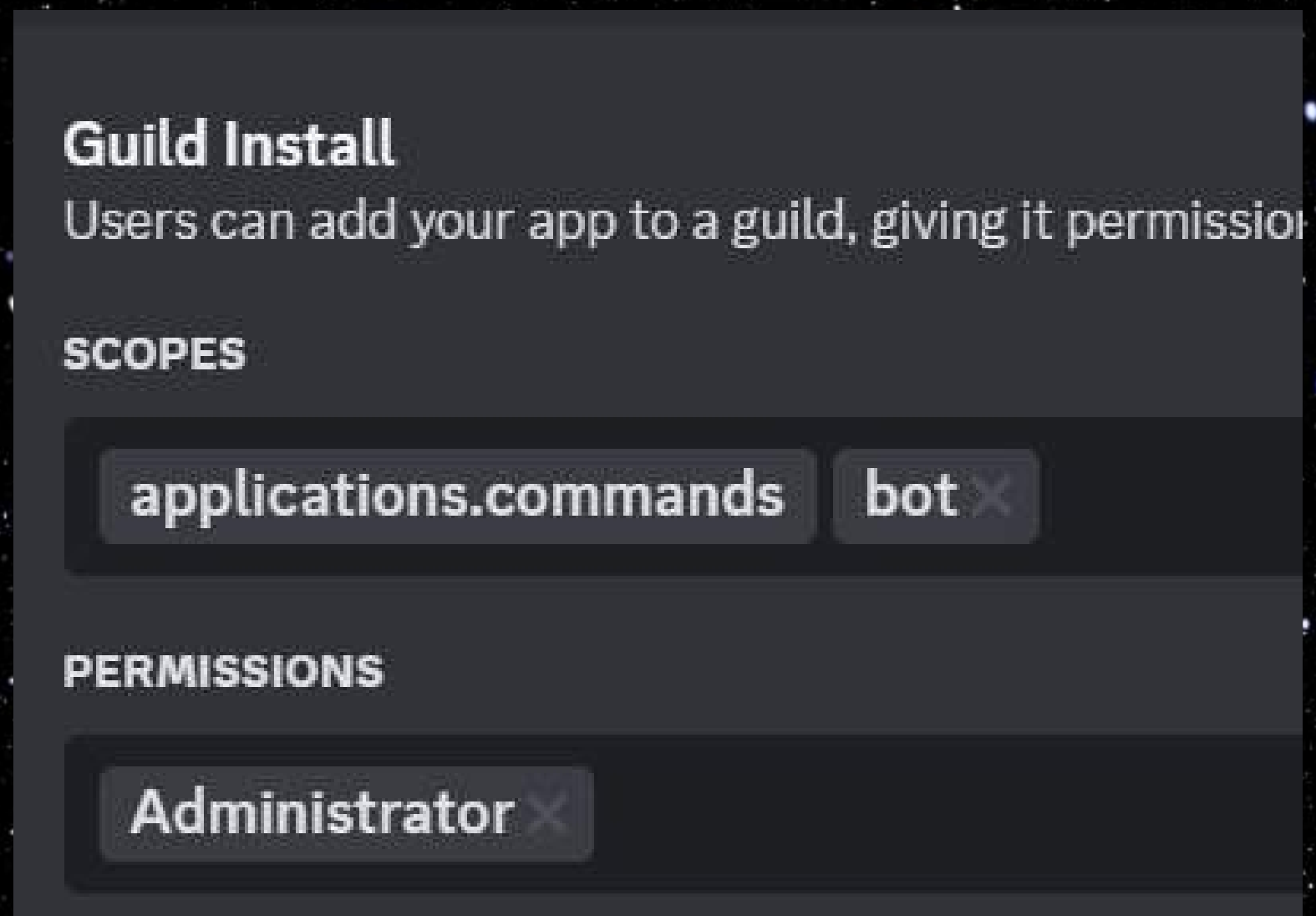
Select

 Careful — you have unsaved changes!

Reset **Save Changes**

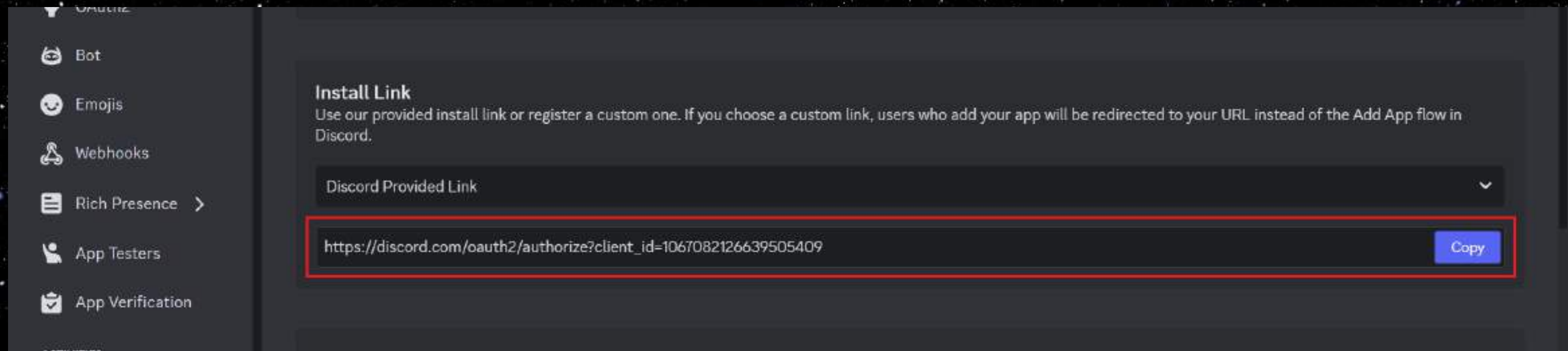
建立 Discord bot

- 在 Guild Install 中的 permissions 欄位輸入 administrator 的選項，



建立 Discord bot

▶ 複製 **Install link** 區塊中的邀請連結



建立 Discord bot

▶ 前往複製的連結，並點選**新增至伺服器**



建立 Discord bot

- ▶ 選擇你剛剛建立的伺服器，把 discord bot 加入到伺服器吧！



建立 Discord bot

▶ 出現這個畫面後，就代表機器人創建成功



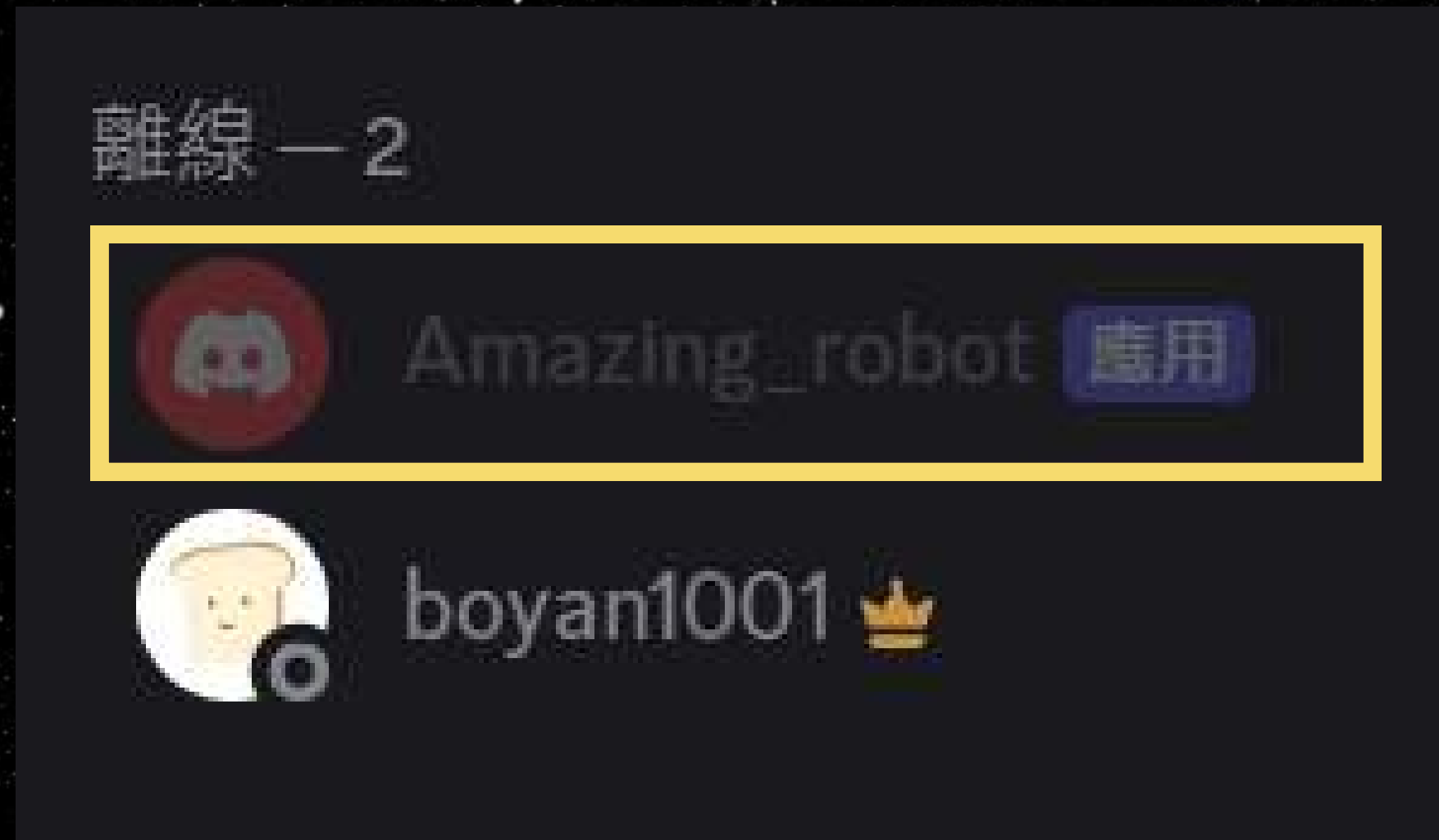
成功！

已授權 **Amazing_robot** 並新增至
Test。

您現在可以關閉此視窗或分頁。

建立 Discord bot

▶ 前往伺服器查看機器人是否存在



設立 Discord bot 的預先準備

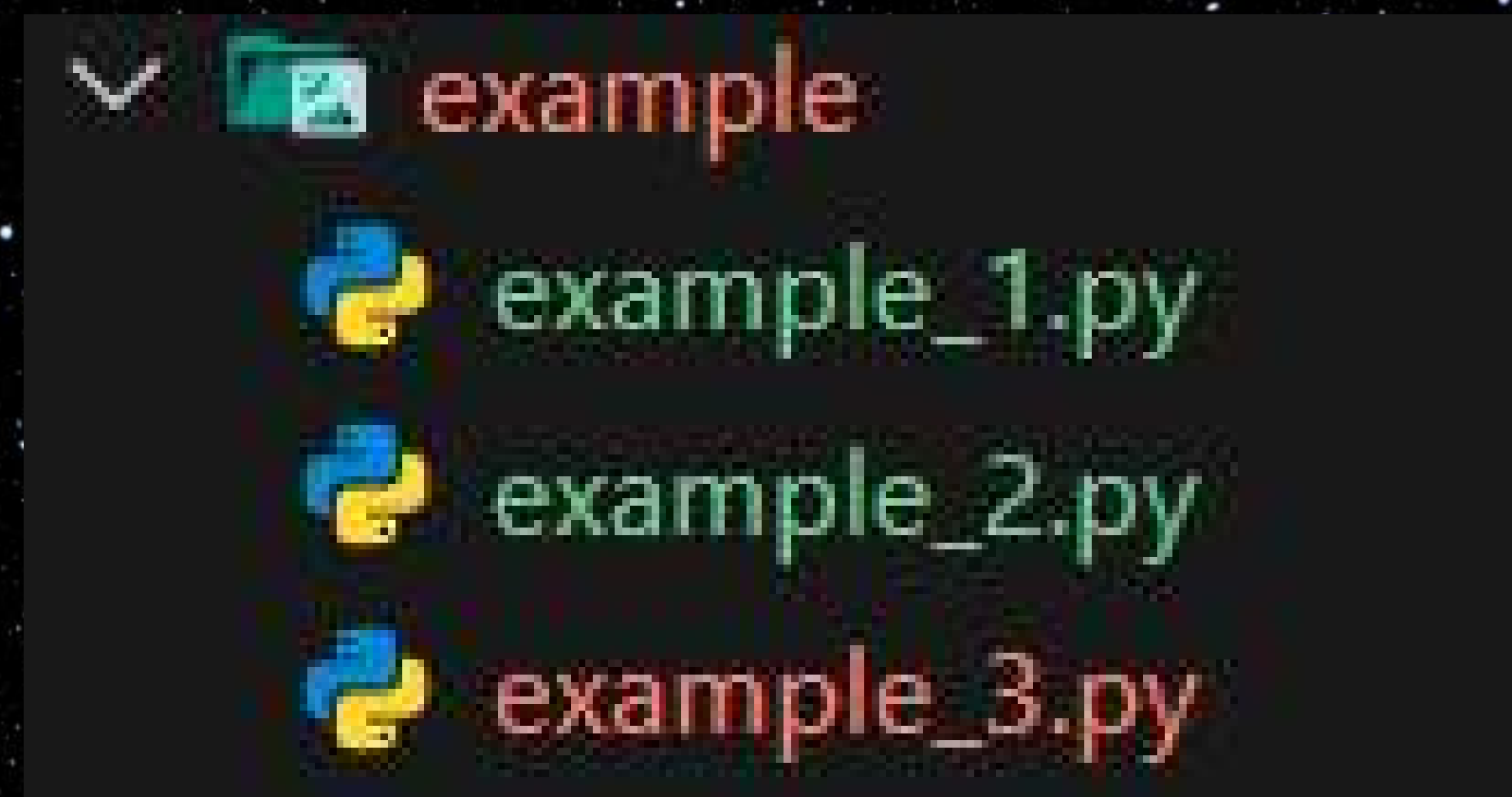
- ▶ 建立 Discord 伺服器
- ▶ 建立 application
- ▶ 撰寫 Discord bot 程式碼

Discord bot! 程式碼

終於要開始寫 code 了，好開心～

打開範例程式碼

- ▶ 請打開放在 `Vscode` 中的範例程式碼
- ▶ 打開 `example_1.py` 檔案



python 套件

- ▶ 套件是指由其他開發者寫的函式庫
- ▶ 可以透過**套件安裝系統**來安裝套件

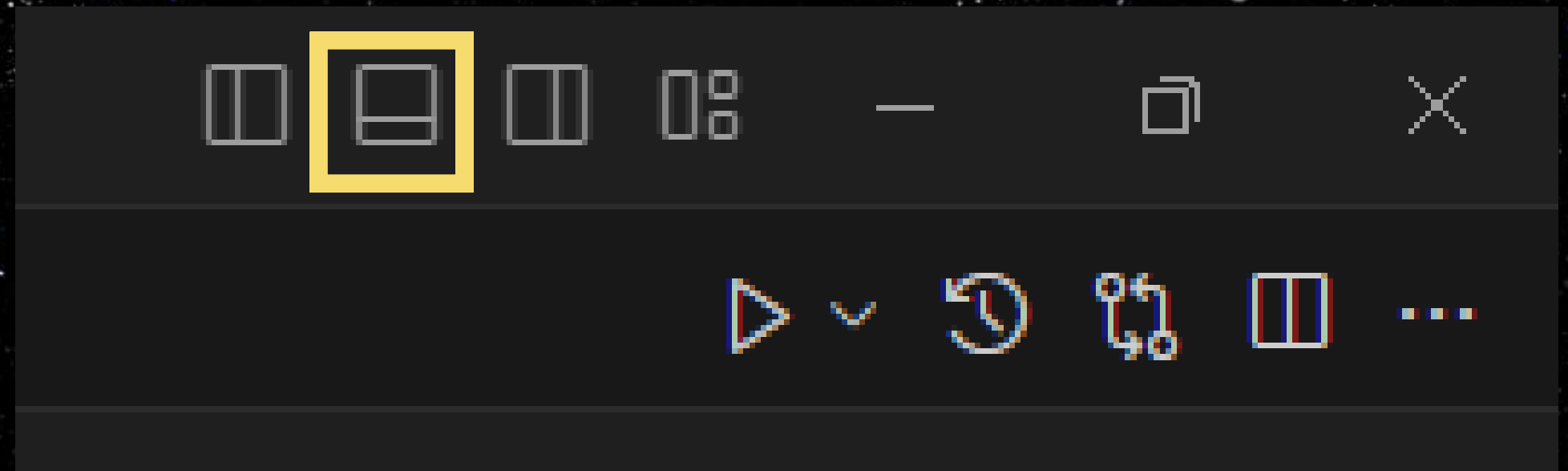
pip 套件系統

- ▶ pip 是 python 的標準套件安裝工具
- ▶ 當我們需要用到 python 原生函式庫以外的函式，就必須透過 pip 安裝**第三方套件**
- ▶ pip 可**安裝**、**升級**、**移除**套件



打開終端機

- ▶ 請在 VScode 中打開終端機
- ▶ 請點擊畫面右上角黃框中的按鈕以顯示終端機



更新 pip

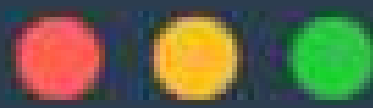
- ▶ 讓我們來更新一下 pip install
- ▶ 請在終端機輸入 `pip install --upgrade pip`



```
pip install --upgrade pip
```


下載相關套件

- ▶ 先在終端機輸入 `discord api (discord.py)`
- ▶ 利用 `pip install` 安裝套件
- ▶ 輸入 `pip install discord.py`



```
pip install discord.py
```

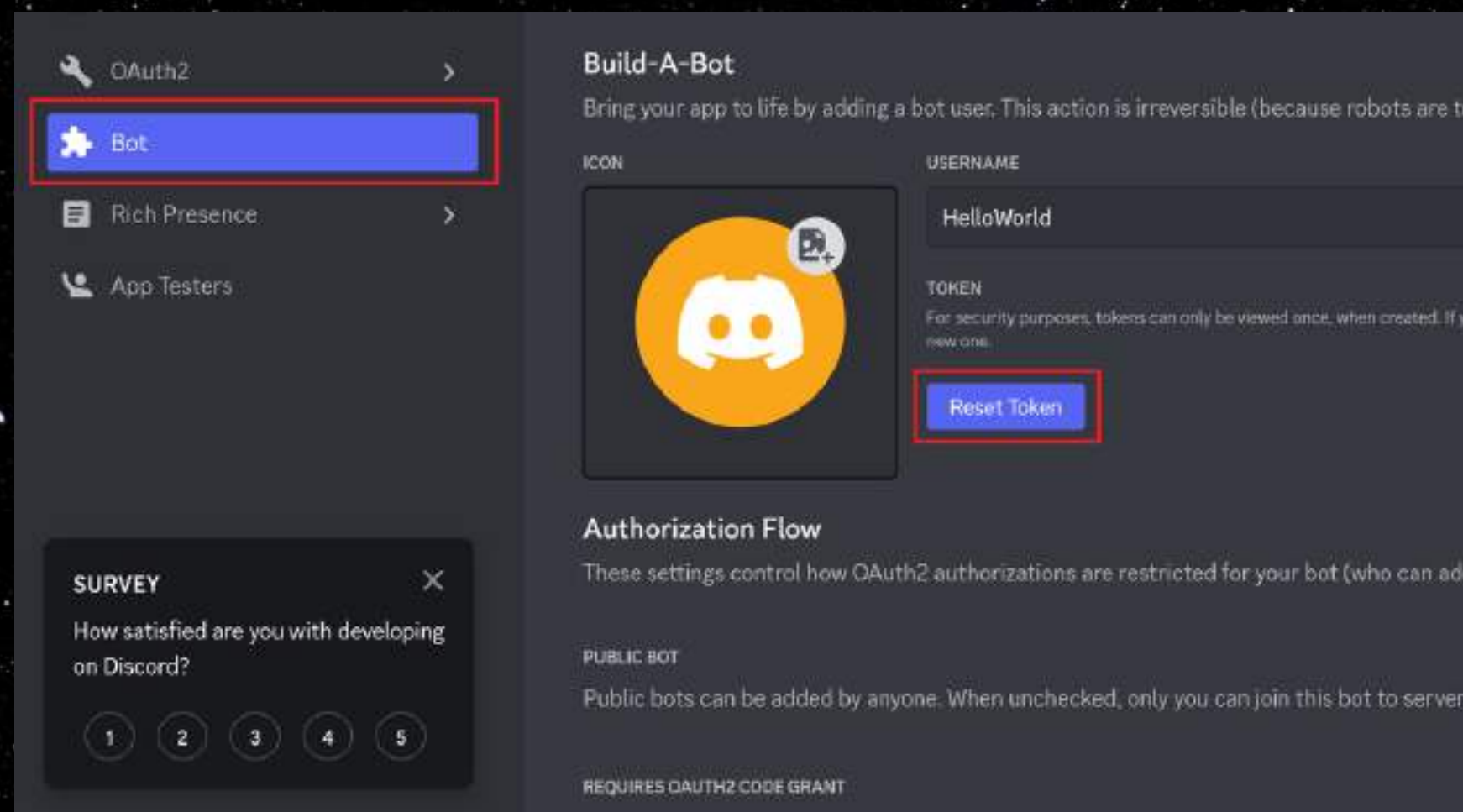

如果要執行程式

- ▶ 點 VScode 右上角的箭頭
旁邊的下拉選單
- ▶ 點執行 Python 檔案



取得 Discord token

➡ 回到 [Discord_Developers](#) 的 Bot 欄位，按下按鈕即可取得 token



取得 Discord token

▶ 點擊 **Yes, do it!** 來重設 Token

RESET BOT'S TOKEN?

Your bot will stop working until you update the token in your bot's code.

Nevermind

Yes, do it!

取得 Discord token

- ▶ 點擊 Copy 即可複製 token
- ▶ Token 是啟用 Discord bot 的鑰匙，擁有者就可以操控機器人，請務必保管好

TOKEN

For security purposes, tokens can only be viewed once, when created. If you forgot or lost access to your token, please regenerate a new one.

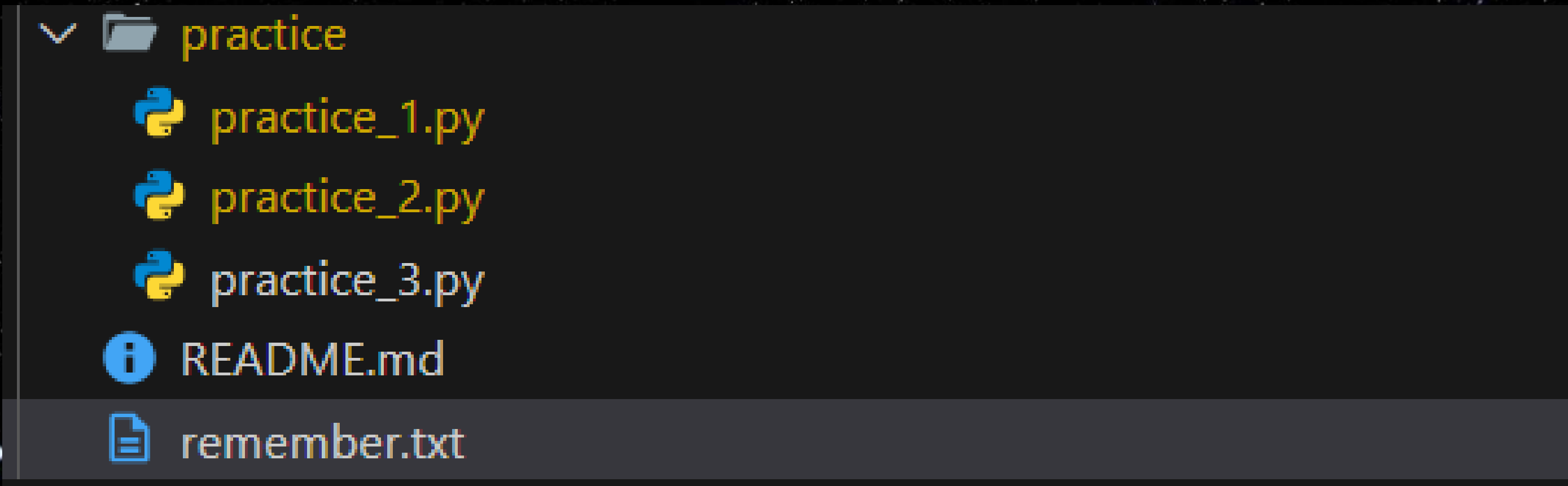
MTM2MjU2ODczMzE0NjQ4MDY4NA.GvJY4W.X25iSQb2YNonUfK_mgBi_NLgGj6ly-639GgFv4

Copy

Reset Token

防止忘記

- ▶ 為了防止你們忘記，可以把你們的 **Discord token** 放到 `remember.txt` 裡面



```
▼ practice
  practice_1.py
  practice_2.py
  practice_3.py
  README.md
  remember.txt
```


執行 Discord bot

- ▶ 將程式中的 `DC_TOKEN` 換成剛剛複製下來的 `token`，然後開始執行



```
dc_token = 'YOUR_DISCORD_TOKEN'
```


設立 Discord bot 的預先準備

- ▶ 建立 Discord 伺服器
- ▶ 建立 application
- ▶ 撰寫 Discord bot 程式碼

執行 Discord bot

- ▶ 當下方出現這一行時，代表執行成功（每個人的 Discord bot 名字#編號）
- ▶ 如果想停止 Discord bot，請直接中止程式運行（在終端機按下 Ctrl + C）

執行 Discord bot

- ▶ 回到你 Discord 伺服器中有機器人在的頻道，輸入 `%Hello` 看看



執行 Discord bot

▶ 好棒耶他回了!!!



歡迎來到 #123 !

這就是 #123 頻道的起點。

 編輯頻道

四月 18, 2020



boyan1001 下午 05:09

%Hello



Amazing_robot 應用 下午 05:09

Hello, world!

停止 Discord bot

- ▶ 如果想要停止 Discord bot，請強制中斷程式
- ▶ 在終端機按下 `Ctrl + C` 即可停止程式

恭喜你們做完第一隻
Discord bot 了~

但是，這節課就這樣結束嗎？

怎麼可能阿~

我們可還有很多沒上到耶

這節課你將學到

- ▶ 生成式 AI 是什麼
- ▶ API 的原理
- ▶ 製作屬於自己的 discord 聊天機器人



Discord bot

程式碼解析

解釋剛剛我們做了什麼

讓我們來看 code

▶ 讓我們來看一下我們的程式碼

```
import discord
import asyncio

from discord.ext import commands

# intents 為機器人的權限，這裡設定為全開
intents = discord.Intents.all()

# bot 是機器人的本體，這邊是設定他前綴，這邊設定為 %
bot = commands.Bot(command_prefix='%', intents=intents, enable_cache=True)
# add enable_cache=True

@bot.event
# 當機器人完成啟動
async def on_ready():
    print(f"目前登入身份 --> {bot.user}")

@bot.command()
# 輸入%Hello呼叫指令
async def Hello(ctx):
    # 回覆Hello, world!
    await ctx.send("Hello, world!")

async def main():
    await bot.start(dc_token)

# 啟動機器人
loop = asyncio.get_running_loop()
wait loop.create_task(main())
```


import 函式庫

- ▶ 這邊是我們 import 必要函式庫的地方
- ▶ discord 是我們的 discord bot
- ▶ asyncio 是非同步處理的套件

```
import discord
import asyncio

from discord.ext import commands
```


Intent 權限

- ▶ Intents 用來設定機器人的權限，這邊設定為全開
- ▶ 這邊的設定會配合前面的 **Privileged Gateway Intents** 所設定的權限，如果前面沒有全打開這個指令會無效

```
# intents 為機器人的權限，這裡設定為全開  
intents = discord.Intents.all()
```

Privileged Gateway Intents

Some [Gateway Intents](#) require approval if your bot is verified. If your bot is not verified, you can toggle those intents below to access them.

PRESENCE INTENT

Required for your bot to receive [Presence Update](#) events.

NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)



SERVER MEMBERS INTENT

Required for your bot to receive events listed under [GUILD_MEMBERS](#).

NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)



MESSAGE CONTENT INTENT

Required for your bot to receive [message content](#) in most messages.

NOTE: Once your bot reaches 100 or more servers, this will require verification and approval. [Read more here](#)



Bot 機器人初始化

- ▶ Bot 用來初始化 Discord bot
- ▶ `command_prefix`：用來標註前綴詞 (prefix)，所有指令呼叫時前面都要加前綴詞
- ▶ `intents`：描述權限 (用剛剛定義好的 `intents`)
- ▶ `enable_cache`：快取功能，可以增加訊息溝通的速度

```
# bot 是機器人的本體，這邊是設定他前綴，這邊設定為 %  
bot = commands.Bot(command_prefix='%', intents=intents, enable_cache=True)
```


@event

- ▶ @event 是事件語法糖，表示下面的 function 是用來偵測現在 Discord bot 所觸發或反映的事件
- ▶ on_ready 負責偵測 discord bot ready 的狀態，並做出 function 內的行為

```
@bot.event
# 當機器人完成啟動
async def on_ready():
    print(f"目前登入身份 --> {bot.user}")
```


@command

- ▶ @command 是指令語法糖，用來表示下面的 function 是 Discord bot 的指令
- ▶ Hello 是使用者自訂的指令

```
@bot.command()  
# 輸入%Hello呼叫指令  
async def Hello(ctx):  
    # 回覆Hello, world!  
    await ctx.send("Hello, world!")
```


async def, await

- ▶ `async def` 是 discord bot 中用來定義程式的方法，相當於 `def`
- ▶ `await` 是 discord bot 中離開函式的方法，類似 `return`，但 `await` 後方一定要接其他函式

```
@bot.command()  
# 輸入%Hello呼叫指令  
async def Hello(ctx):  
    # 回覆Hello, world!  
    await ctx.send("Hello, world!")
```


ctx

- ▶ ctx 是上下文物件，用來指令呼叫的時間點、呼叫地點、呼叫者等
- ▶ ctx.send 是用來將參數的內容回傳訊息給使用者

```
@bot.command()  
# 輸入%Hello呼叫指令  
async def Hello(ctx):  
    # 回覆Hello, world!  
    await ctx.send("Hello, world!")
```


bot.start

- ▶ bot.start 用來啟動 Discord bot，輸入 Discord bot 的 token 就可以啟動了
- ▶ 下面的東西是用來啟動機器人的函式

```
async def main():  
    await bot.start(dc_token)  
  
# 啟動機器人  
loop = asyncio.get_running_loop()  
await loop.create_task(main())
```


@event+

事件語法糖介紹

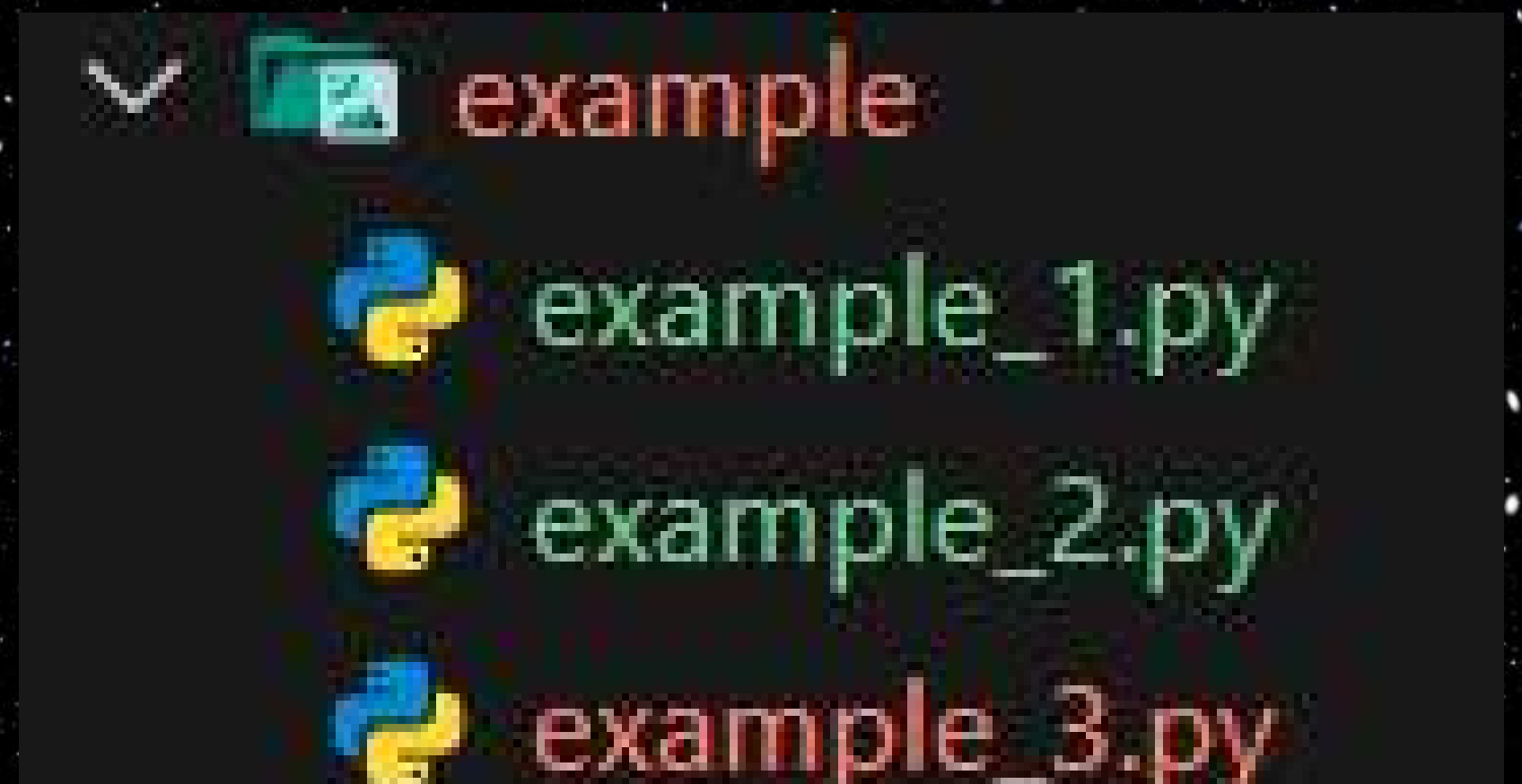
基礎 Discord bot 教程

@event

- ▶ 事件（Event）是指某些特定的時刻或情境下，系統或應用程序會自動觸發的通知或反應
- ▶ 在 Discord bot 中，當事件在 Discord 伺服器中發生，Discord API 會向 Bot 發送通知，開發者可以根據這些事件來執行相對應的操作。
- ▶ 事件的函數名是固定的

@event

▶ 請打開 `example_2.py`



@event

- ▶ 舉例來說，`on_ready()` 函式就是一個事件，當 Discord Bot 完成準備階段時會觸發這個事件

```
@bot.event
# 當機器人完成啟動
async def on_ready():
    print(f"目前登入身份 --> {bot.user}")
```


Messages 訊息

- ▶ `on_message()` : 發送訊息
- ▶ 當用戶在伺服器中發送新訊息時觸發這個事件

```
@bot.event
async def on_message(message):
    await message.channel.send(f"收到訊息: {message.content} (來自: {message.author.name})")
```


Messages 訊息

- ▶ `message.channel.send` : 傳送訊息回頻道中
- ▶ `message.content` : `on_message` 擷取到的訊息

```
@bot.event
async def on_message(message):
    await message.channel.send(f"收到訊息: {message.content} (來自: {message.author.name})")
```


兩種輸出的差別

- ▶ `print` : 顯示在終端機
- ▶ `message.channel.send` : 發送訊息到頻道裡

```
@bot.event
async def on_message(message):
    # print -> 顯示在終端機
    print(f"收到訊息: {message.content} (來自: {message.author.name})")

    # await message.channel.send -> 發送訊息到頻道
    await message.channel.send(f"收到訊息: {message.content} (來自: {message.author.name})")
```


設立訊息回傳限制

- ▶ 利用 `message.author` 來判斷訊息的作者
- ▶ `bot.user`：機器人本身

```
if message.author == bot.user:  
    # 機器人不回覆自己的訊息  
    return
```

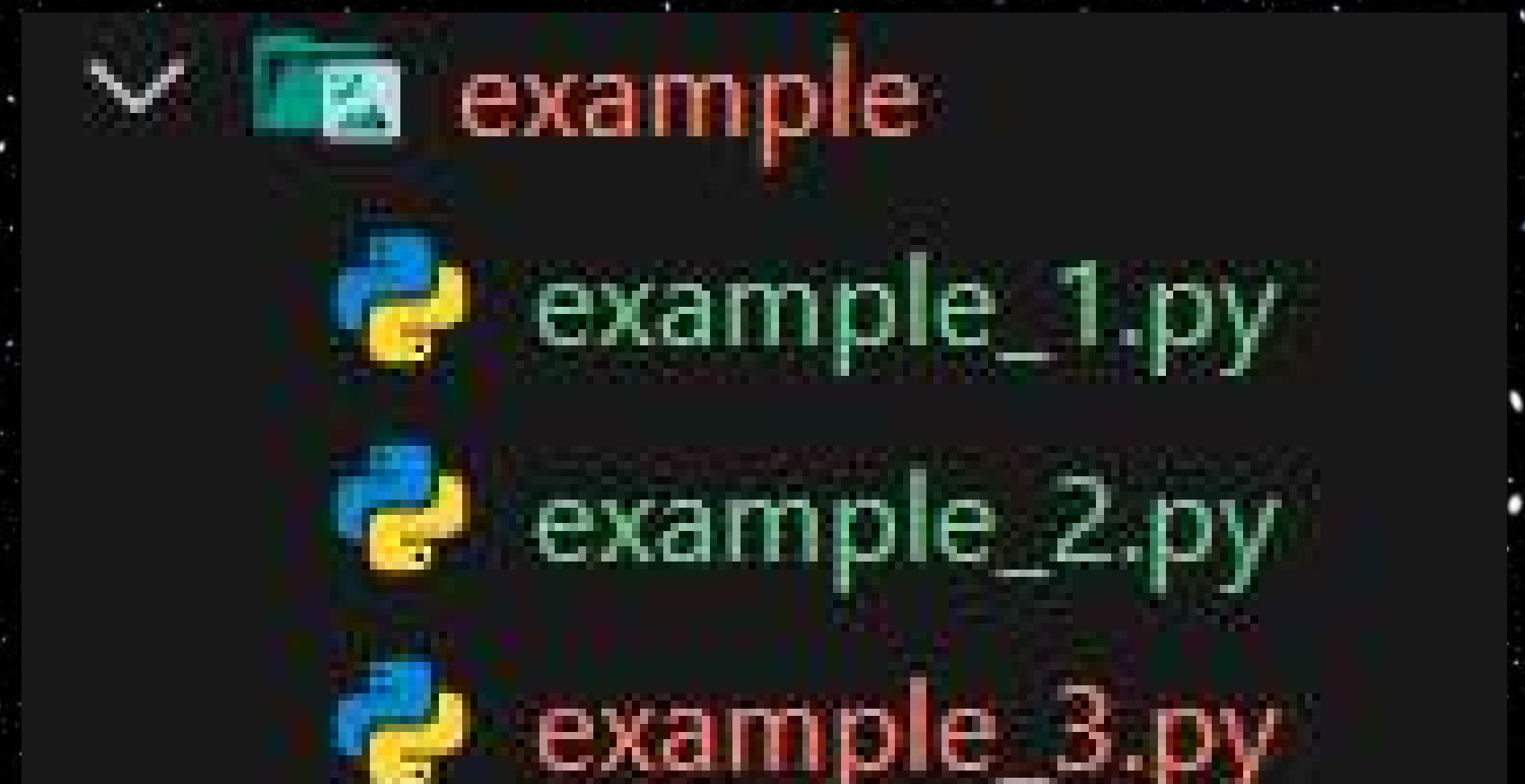

@command

命令語法糖介紹

基礎 Discord bot 教程

@command

▶ 請打開 `example_2.py`



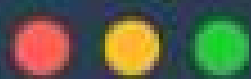
@command

- ▶ 指令 (command) 是由程式設計者自訂，必須透過 `ctx` 與伺服器的使用者互動
- ▶ 在伺服器中需要前綴 + 函式名才能呼叫函式
- ▶ 例如：這邊必須要輸入 `%Hello` 才有回應

```
@bot.command()  
# 輸入%Hello呼叫指令  
async def Hello(ctx):  
    # 回覆Hello, world!  
    await ctx.send("Hello, world!")
```


@command

▶ 前綴可由下方這個指令來控制



```
# bot 是機器人的本體，這邊是設定他前綴，這邊設定為 %  
bot = commands.Bot(command_prefix='%', intents=intents)
```


ctx

- ▶ ctx 是上下文物件，用來指令呼叫的時間點、呼叫地點、呼叫者等
- ▶ ctx.send 是用來將參數的內容回傳訊息給使用者

```
@bot.command()  
# 輸入%Hello呼叫指令  
async def Hello(ctx):  
    # 回覆Hello, world!  
    await ctx.send("Hello, world!")
```

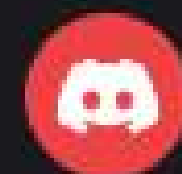

@command

- ▶ 如果我們同時輸入 `arg` 這個變數，`arg` 可以幫我們處理呼叫詞後面的字串，注意，呼叫詞與 `arg` 中間要留空格，並且 `arg` 預設型別是 `str`
- ▶ 例如：`%Hello` 我肚子很餓

```
@bot.command()  
# 輸入%Hello呼叫指令  
async def Hello(ctx, arg):  
    await ctx.send(arg)
```



Toast 下午 07:01
%Hello 我今天很好



Amazing_robot 應用 下午 07:01
我今天很好

@command

- ▶ 當然，你也可以輸入多個變數，並且可以指定變數的型別
- ▶ 同樣地，每個變數之間都要留空格

```
@bot.command()  
async def add(ctx, a: int, b: int):  
    await ctx.send(f"{a} + {b} = {a + b}")
```


練習時間~

起床了~別睡了該練習了!!

實作

請寫出有以下功能的 Discord bot 吧！

- ▶ 使用 # 作為前綴詞，製作可以進行兩個小數四則運算的計算機（額外挑戰：實作更多功能，可自由發揮）
- ▶ 幫你的機器人寫一個自我介紹，輸入 Hello 後可以回傳

請打開 `practice_1.py` 進行實作

實作

▶ 參考答案如下：

```
@bot.event
async def on_reaction_add(reaction, user):
    await reaction.message.channel.send(f"{user.name} 偷偷按下了 {reaction.emoji}")

@bot.command()
async def Hello(ctx):
    await ctx.send(f"Hello, {ctx.author.name}！我是你的機器人助手！")

@bot.command()
async def add(ctx, a: float, b: float):
    await ctx.send(f"{a} + {b} = {a + b}")

@bot.command()
async def subtract(ctx, a: float, b: float):
    await ctx.send(f"{a} - {b} = {a - b}")

@bot.command()
async def multiply(ctx, a: float, b: float):
    await ctx.send(f"{a} * {b} = {a * b}")

@bot.command()
async def divide(ctx, a: float, b: float):
    if b == 0:
        await ctx.send("除數不能為零！")
    else:
        await ctx.send(f"{a} / {b} = {a / b}")
```


下課 10 分鐘

終於下課了？太棒了！

上課了

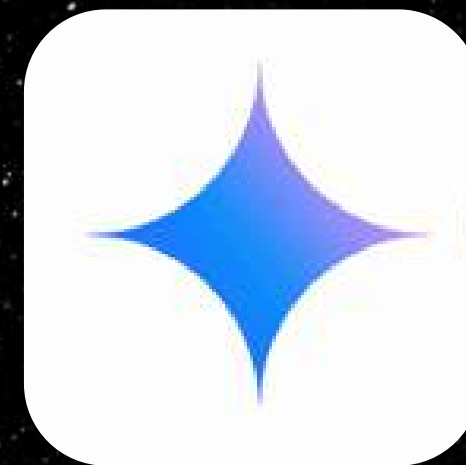
快樂的時光總是特別快QQ

生成式 AI 是什麼？

神奇的 ChatGPT 是如何運作的？

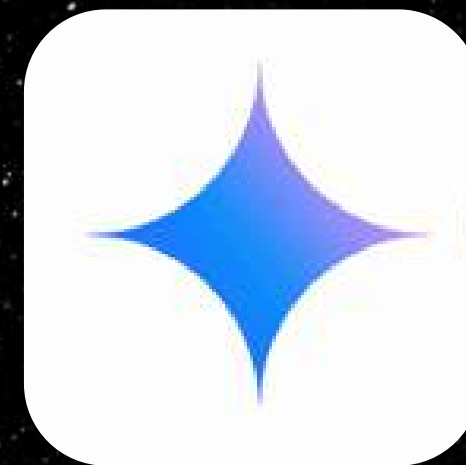
什麼是生成式 AI

- ▶ 是一種人工智慧
- ▶ 可以根據輸入的 Prompt 的內容，自動生成新的內容
- ▶ 輸入輸出類型多元，文字、圖片、影片皆可



Prompt 是什麼

- ▶ Prompt 是指提示詞
- ▶ 我們輸入 Prompt 給生成式 AI，生成式 AI 回傳給我們結果



Gemini

- ▶ 是一款由 Google 開發的生成式 AI
- ▶ 大部分模型皆為免費
- ▶ 本課程將以 Gemini 為主
- ▶ [Gemini_連結](#)



生成式 AI 的應用

- ▶ 聊天機器人
- ▶ 翻譯工具
- ▶ 自動寫作
- ▶ 程式碼生成
- ▶ 醫療診斷輔助

生成式 AI 的缺點

- ▶ 產生出來的答案不一定正確

請問 1.11 與 1.6 哪一個比較大

◆ 1.11 比較大。



API

原理介紹

api 是什麼？那能吃嗎www

什麼是API

- ▶ API 是指應用程式介面 (Application Programming Interface)
- ▶ 是應用程式之間的橋樑，把現有的技術、資料開放出來，讓別人能夠二次使用，以減少開發成本

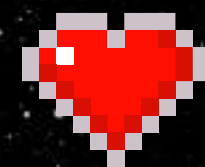
API的原理



API的原理

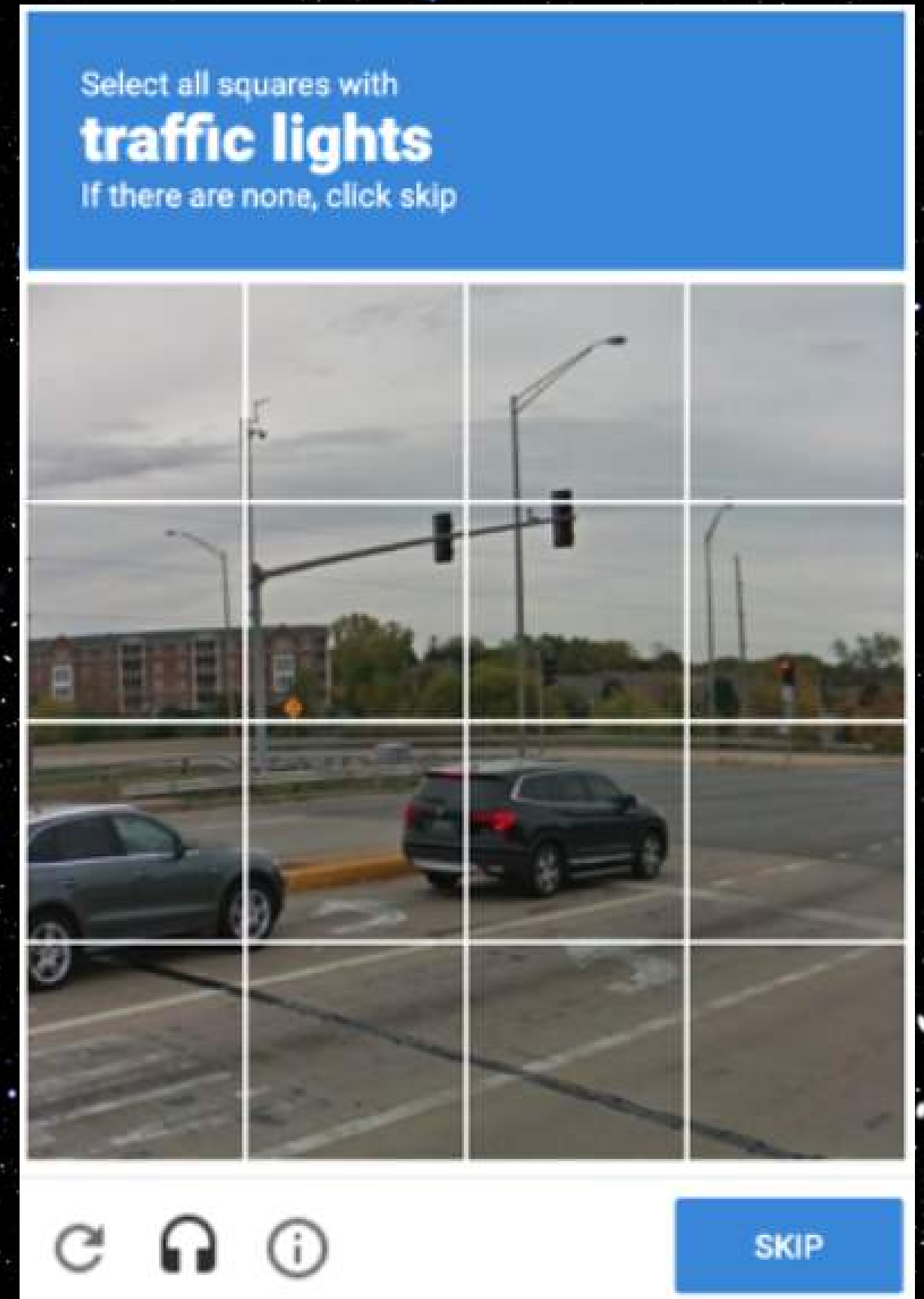


API 的種類



特色 API

- ▶ 使用某個**特定**的應用程式，來解決**特定**問題
- ▶ e.g. reCAPTCHA



API 的種類

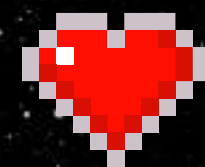
♥ 資料 api

▶ 呼叫另一個應用程式，輸出某些**特定**
的資料

▶ e.g. 政府資料開放
平臺

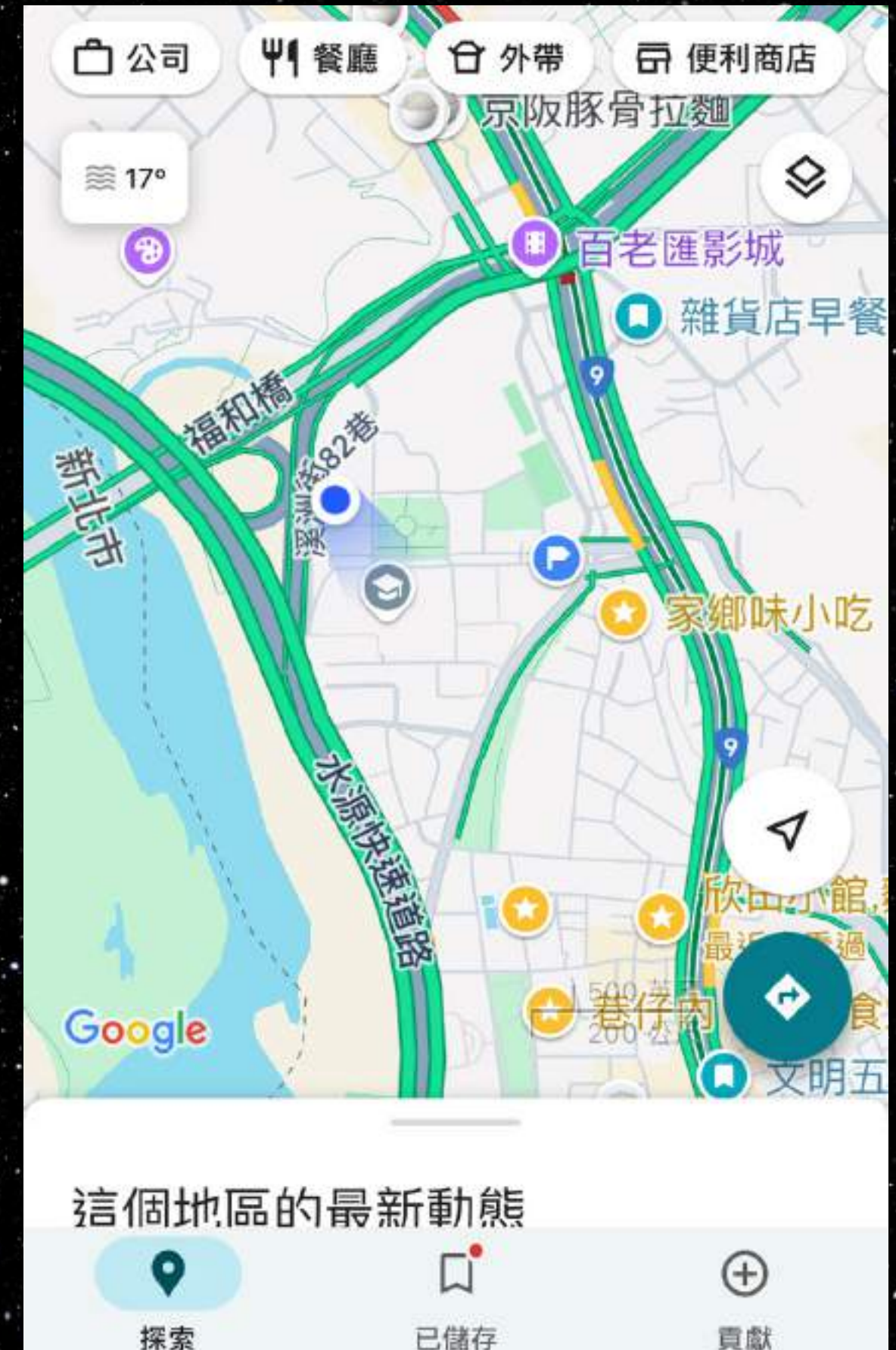


API 的種類



硬體 api

- ▶ 可以操作硬體的 api，通常與作業系統有關
- ▶ e.g. 手機定位的 API



Gemini api

實作

用程式跟 Gemini 對話吧！



Gemini api

- ▶ 是可以透過程式與 Gemini 接觸的 api
有提供免費的模型
- ▶ 打開 example_4.py



安裝環境

- ▶ 在終端機輸入以下指令：
- ▶ `pip install -q -U google-generativeai`



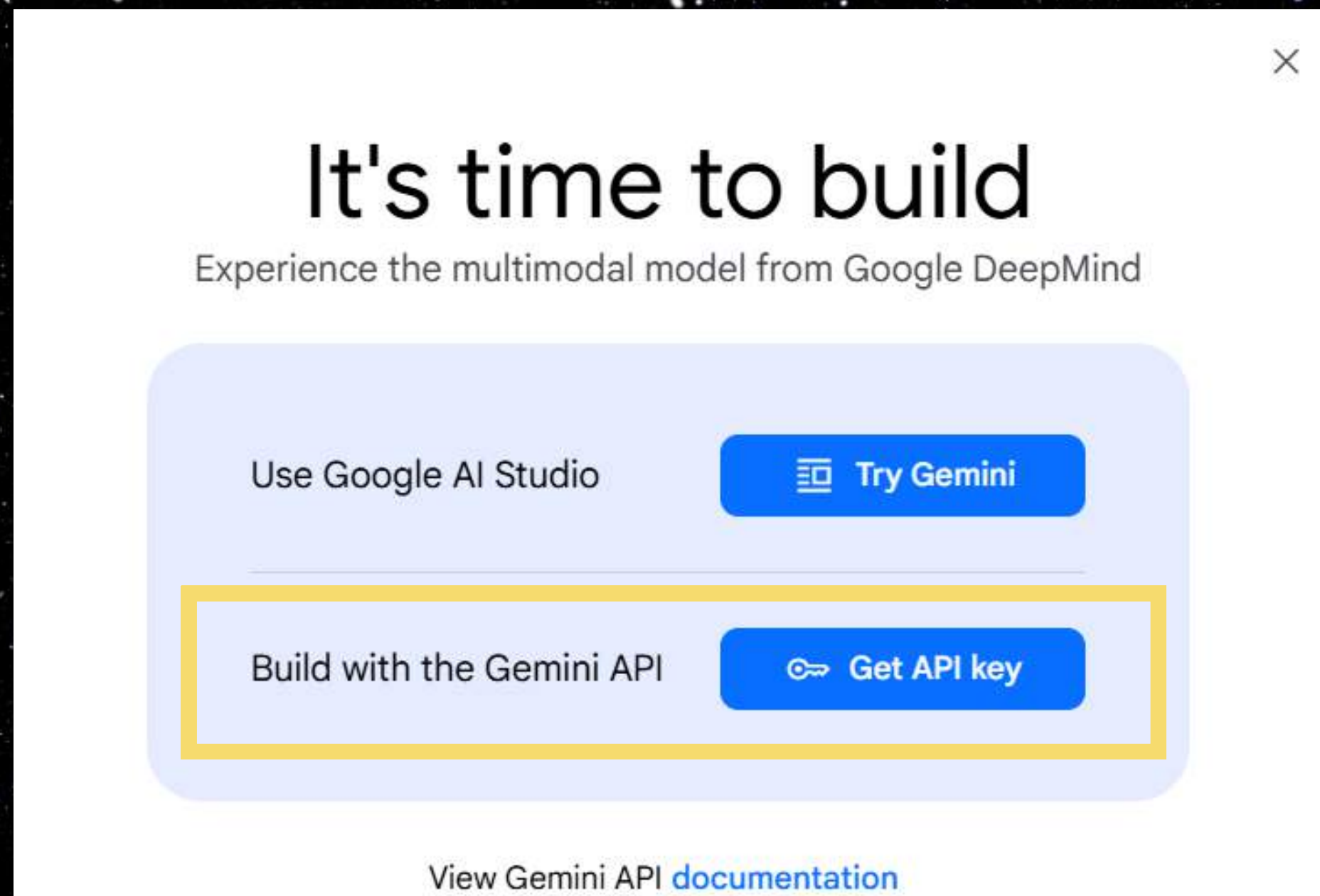
```
pip install -q -U google-generativeai
```


取得 Gemini api key

- ▶ 必須要取得 Gemini api 的金鑰，我們才能使用 Gemini api
- ▶ 請前往以下網址取得金鑰：[連結](#)
- ▶ 建議不要使用學校帳號

取得 Gemini api key

▶ 點擊下方 Get API key



取得 Gemini api key

▶ 點擊 **Create API key** 來取得自己的 API 金鑰

API Keys

Quickly test the Gemini API

[API quickstart guide](#)

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?key=GEMINI_API_KEY" \  
-H 'Content-Type: application/json' \  
-X POST \  

```

+ Create API key

取得 Gemini api key

- ▶ 如果之前沒有使用過 Google Cloud 的話，會跑出這個錯誤訊息
- ▶ 請點擊中間的藍字 [Google Cloud Console](#)

Unable to create API key

We were unable to create an API key and Google Cloud project for you. Please create a project in the [Google Cloud Console](#).

Close

取得 Gemini api key

- ▶ 勾選服務條款
- ▶ 點選同意並繼續



The screenshot shows the Google Cloud 'Welcome' page. At the top is the Google Cloud logo. Below it is a welcome message: '，歡迎使用！' (Welcome!). This is followed by a description: '在單一位置建立及管理 Google Cloud 執行個體、磁碟、網路和其他資源。' (Build and manage Google Cloud VMs, disks, networks, and other resources in a single location). There is a green circular profile icon with the letter 'P' and a '切換帳戶' (Switch account) link. A '國家/地區' (Country/Region) dropdown menu is set to '台灣' (Taiwan). Under the '服務條款' (Terms of Service) section, there is a checked checkbox and the text: '我同意《[Google Cloud Platform 服務條款](#)》和[任何適用服務與 API 的服務條款](#)。' (I agree to the Google Cloud Platform Terms of Service and any applicable service and API Terms of Service). At the bottom right, there is a yellow button labeled '同意並繼續' (Agree and continue).

同意並繼續

取得 Gemini api key

- ▶ 專案名稱可自訂，位置不要調
- ▶ 點選**建立**

新增專案



您的配額還可供建立 12 projects。建議您要求增加配額或刪除專案。
[瞭解詳情](#)

[Manage Quotas](#)

專案名稱 *

My Project 3361



專案 ID：coherent-window-463209-h6。ID 設定後即無法變更。 [編輯](#)

位置 *

無組織

[瀏覽](#)

上層機構或資料夾

建立

取消

到這個畫面就成功了

 領取價值 \$300 美元的抵免額，開始免費試用。請放心，即便抵免額用盡也不會產生費用。[瞭解詳情](#)

關閉

免費試用

 Google Cloud

My Project 3361

搜尋資源、文件、產品和其他項目 (請按 / 鍵)

搜尋











P

資訊主頁

活動

推薦內容

自訂

 專案資訊

專案名稱
My Project 3361

專案編號
916474190624

專案 ID
coherent-window-463209-h6

將使用者新增至這項專案

前往專案設定

 資源

BigQuery
資料倉儲/分析

SQL
代管的 MySQL、PostgreSQL、SQL Server

Compute Engine
VM、GPU、TPU、磁碟

API API

要求 (每秒要求數)

1.0

0.8

0.6

0.4

0.2

0

4:30 4:45 5 下午 5:15

所選時間範圍沒有可用資料。

前往 API 總覽

 Google Cloud Platform 服務狀態

所有服務狀態皆正常

前往 Cloud 狀態資訊主頁

 Monitoring

建立我的資訊主頁

設定警告政策

建立運作時間檢查

查看所有資訊主頁

前往 Monitoring

API Error Reporting

取得 Gemini api key

▶ 返回剛剛的這個頁面 ([連結](#))，按下 **Create API key**

API Keys

Quickly test the Gemini API

[API quickstart guide](#)

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?key=GEMINI_API_KEY" \  
-H 'Content-Type: application/json' \  
-X POST \  

```

+ Create API key

取得 API 金鑰

▶ 選取剛剛建立的專案

Create API key

×

Select a project from your existing Google Cloud projects

Search Google Cloud projects

Q |

My Project 3361 coherent-window-463209-h6

Or

Create API key in new project

取得 API 金鑰

▶ 按下 Create API key in existing project

Create API key

×

Select a project from your existing Google Cloud projects

Search Google Cloud projects

Q My Project 3361

coherent-window-463209-h6

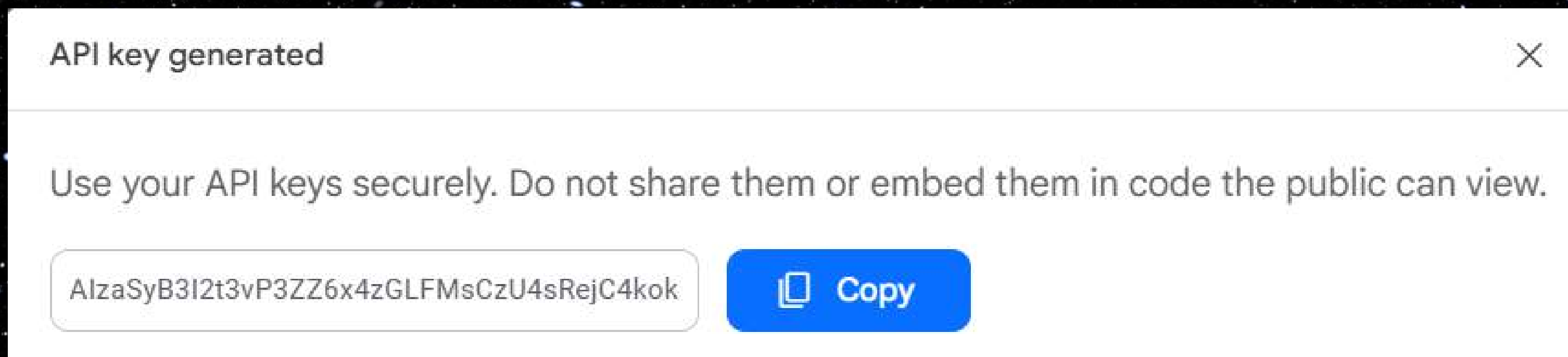
Create API key in existing project

Or

Create API key in new project

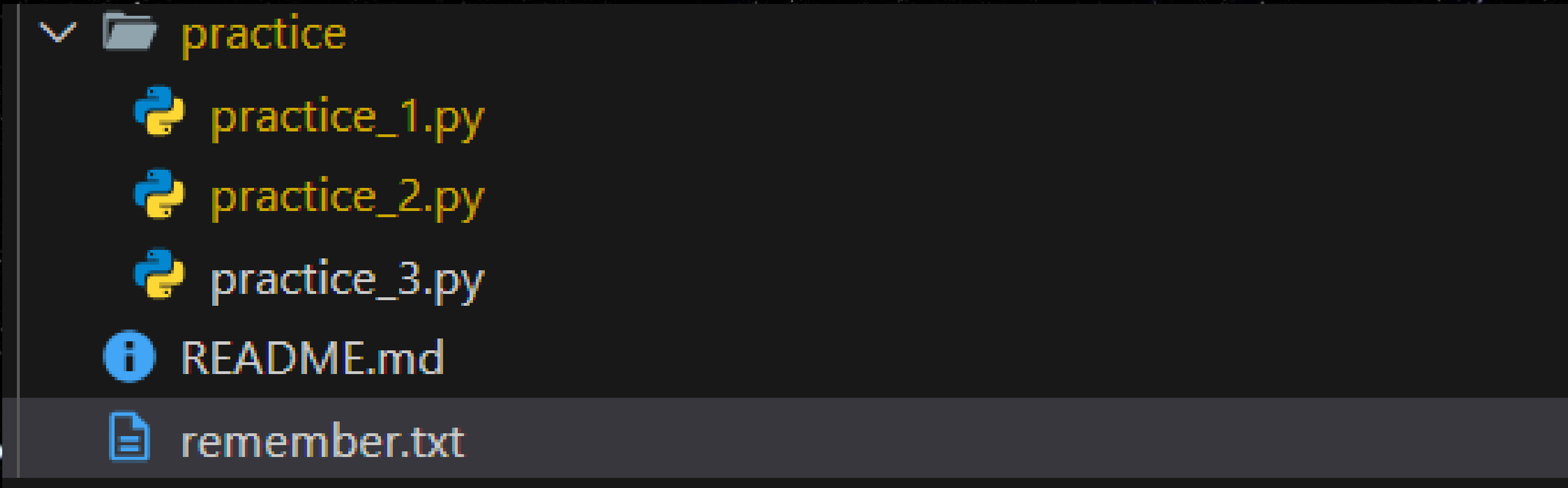
取得 API 金鑰

- ▶ 出現此畫面代表生成 API 金鑰成功
- ▶ 按下 **copy** 可以複製金鑰，請把金鑰保管好



防止忘記

- ▶ 為了防止你們忘記，可以把你們的 `gemini api key` 放到 `remember.txt` 裡面



```
▼ practice
  practice_1.py
  practice_2.py
  practice_3.py
  README.md
  remember.txt
```


取得 API 金鑰

- ▶ 打開 `example_4.py`
- ▶ 把檔案中的 `GEMINI_API_KEY` 替換成你自己的 `Gemini API 金鑰`
- ▶ 測試看看是否可以順利運行，如果能跑出對話就代表成功了！

```
csiecamp_samplecode ➤ python gemini_api_learning.py
```

- 我是一個大型語言模型，由 Google 訓練。

Gemini api

程式碼解析

解釋剛剛我們做了什麼

system 程式碼

- ▶ 引入 `google.generativeai` 函式庫
- ▶ 輸入自己的 `gemini_api_key` 並啟動 `gemini`
- ▶ 選擇使用的模型（本課程請全程使用 `gemini-2.0-flash`）

```
import google.generativeai as genai

# 填入你的 gemini 的 API key
GEMINI_API_KEY = "YOUR_GEMINI_API_KEY"

# 啟動 gemini
genai.configure(api_key=GEMINI_API_KEY)

# 設定模型
model = genai.GenerativeModel('gemini-2.0-flash')
```


user 程式碼

- ▶ `prompt` : 要問 gemini 的問題
- ▶ `response` : gemini 的輸出
- ▶ gemini 的回答放在 `response.text` 中

```
prompt = "請問你是誰?"  
response = model.generate_content(prompt)  
print(response.text)
```


兩者區別

- ▶ `system` 程式碼區：**模型的相關設定**，可套用於所有對話，程式執行優先性較高
- ▶ `user` 程式碼區：每個對話的輸入與輸出，僅適用於一則對話，程式執行優先性較低

練習時間～

大家跟我一起寫程式

風格化輸出

讓 gemini 變成我們想要的樣子

方法一：加在 Prompt 裡

- ▶ 打開 `example_5.py`
- ▶ 直接在 `prompt` 前面加上自己想要的風格

```
prompt_style = "你是一位在大學開設程式設計課程的教授，你的名字叫做：nekocat，你已經教學五年了，你是位和善的 40 歲大叔，請用和善且專業的語氣回答以下問題："

```

```
prompt = input("請輸入你的問題：")

```

```
prompt = prompt_style + prompt

```

```
response = model.generate_content(prompt)

```


方法二：系統指令

- ▶ 打開 `example_6.py`
- ▶ 使用 `system_instruction` 來，直接把風格內嵌入 `model` 宣告中

```
model = genai.GenerativeModel(  
    'gemini-2.0-flash',  
    system_instruction="你是在女僕咖啡廳工作的女僕，請用可愛的語  
    氣回答以下問題，你的名字吐司醬，請在你說話的字尾記得加上波浪號哦"  
)  
  
prompt = ""  
response = model.generate_content(prompt)  
print(response.text)
```


兩者區別

- ▶ user prompt（方法一）：強制性較低，僅會影響該次對話
- ▶ system instruction（方法二）：強制性高，可套用於所有對話
- ▶ 實務上我們通常會用方法二

記憶

讓 gemini 不要那麼健忘

history

- ▶ 我們可以利用 Gemini 內建的 `chat` 模式來讓 gemini 記憶我們聊過的內容為何
- ▶ 利用 `start_chat` 來初始化對話記憶
- ▶ 打開 `example_7.py` 來查看內容

```
chat = model.start_chat(history=[])  
response = chat.send_message(prompt)
```


history

history 本身是字典 (dict) 結構，負責記錄之前的對話

```
parts {  
  text: "你好我是土司很高興認識你"  
}  
role: "user"  
  
parts {  
  text: "嗨，土司！很高興認識你。很高興能與你聊天。你有什麼想聊的嗎？  
我可以幫你什麼忙？\n"  
}  
role: "model"
```


練習時間~

起床了~別睡了該練習了!!

實作

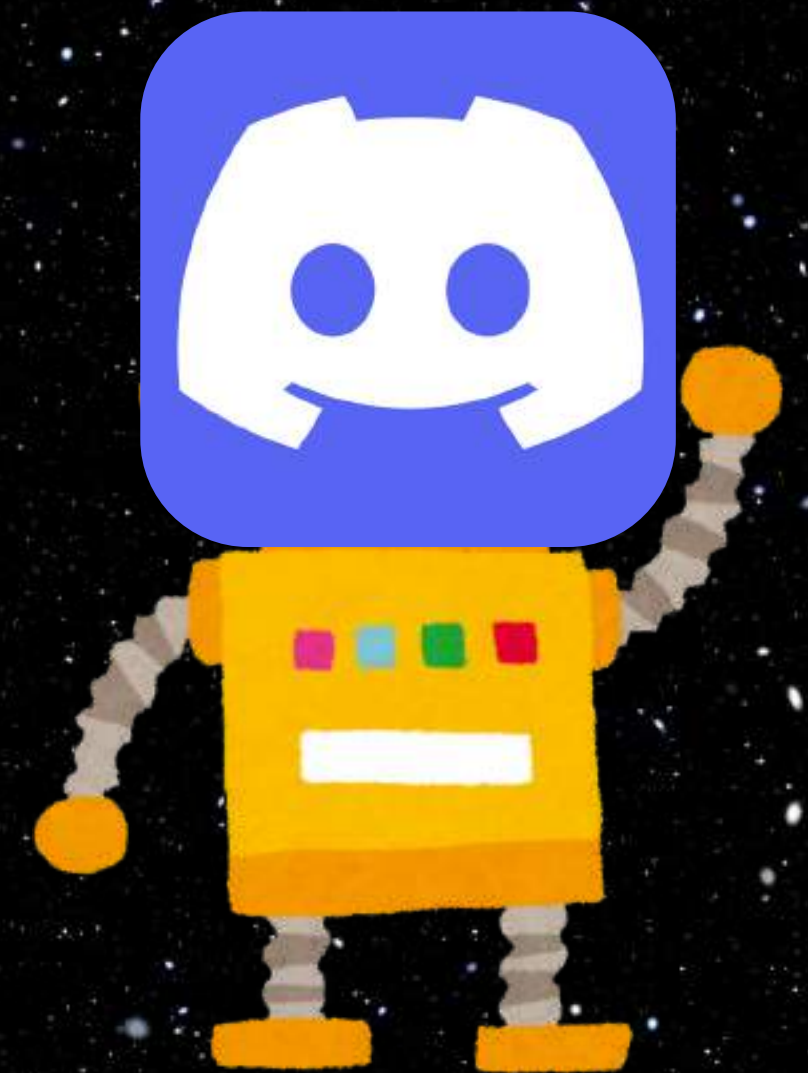
請使用剛剛學到的東西，嘗試讓機器人產生以下不同風格的回答吧～

- ▶ 一位名為 nekocat 貓娘，可愛中帶有傲嬌
- ▶ 一個 90 歲參與過二戰的老兵，語氣中帶有參戰過的自信，然而年事已高說話有點結巴
- ▶ 記得要提供反覆輸入與記憶的功能

請在 `practice_2.py` 中完成

最終 Discord bot 實作

讓機器人跟你在 Discord 對話吧



概念

- ▶ 利用 `command` 取得想輸入的話，再利用剛剛學到的 `gemini api` 對話處理方法，就可以在 Discord 裡面跟 Gemini 對話了
- ▶ 相關 code 可以參考 `example_8.py`

實作時間～

加油～～

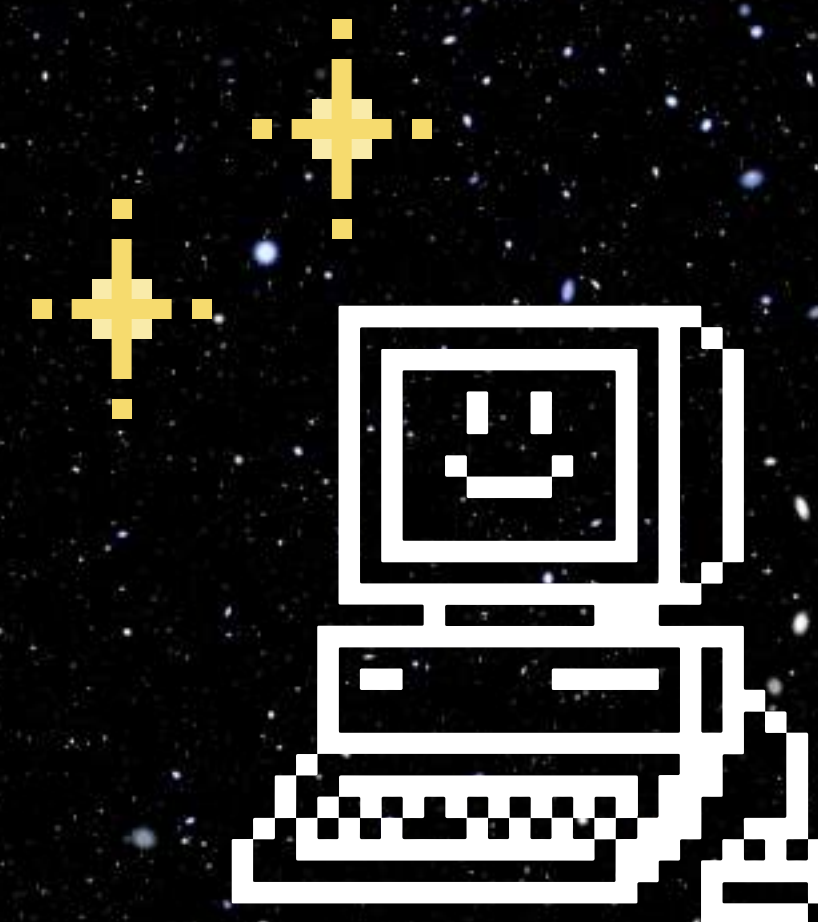
實作

請使用這結課學到的東西，做出一個你最喜歡的聊天機器人吧～

- ▶ 寫出你想要寫出來的機器人吧，有問題可以找我或助教
- ▶ 要實作記憶的功能
- ▶ 請在 `practice_3.py` 修改

這節課學到了

- ▶ 生成式 AI 是什麼
- ▶ API 的原理
- ▶ 製作屬於自己的 discord 聊天機器人





謝謝大家

還有任何問題可以來問我哦～